ITA0615 – MACHINE LEARNING
ASSIGNMENT 5

By:

Harshith G

192525394

Q1 – Represent and Compute with Sensor Data Using NumPy Arrays

Step 1 – Create the array and report its attributes

The table is entered as a list of rows (one row per machine, one column per day) and converted into a two-dimensional NumPy array. Its attributes are then reported and the array is converted to float32 to compare memory.

Python code

```
import numpy as np

temps = np.array([[70, 72, 75, 71, 90],
                  [65, 66, 64, 67, 66],
                  [80, 82, 85, 88, 91],
                  [60, 61, 59, 62, 60]])
print(temps)
print("shape    :", temps.shape)
print("ndim     :", temps.ndim)
print("size     :", temps.size)
print("dtype    :", temps.dtype)
print("itemsize :", temps.itemsize, "bytes per element")
print("nbytes   :", temps.nbytes, "bytes")

t32 = temps.astype(np.float32)
print("float32 dtype :", t32.dtype, "| nbytes :", t32.nbytes, "bytes")
print("Memory saved  :", temps.nbytes - t32.nbytes, "bytes (",
      round(100 * (1 - t32.nbytes / temps.nbytes)), "% )")
```

Output

```
[[70 72 75 71 90]
 [65 66 64 67 66]
 [80 82 85 88 91]
 [60 61 59 62 60]]
shape: (4, 5)
ndim: 2
```

```
size: 20
dtype: int64
itemsize: 8 bytes per element
nbytes: 160 bytes
float32 dtype: float32 | nbytes: 80 bytes
Memory saved: 80 bytes (50 %)
```

The array has 4 rows (machines) and 5 columns (days): 20 elements, 2 dimensions. The default integer type on this system is int64 (8 bytes per element, 160 bytes in total). float32 uses 4 bytes per element, so the memory is halved to 80 bytes. The saving is useful for large sensor arrays, at the cost of lower numerical precision (about 7 significant digits).

Step 2 – Create the same data in different ways and prove equality

The same data is built (a) from a flat Python list with reshape, (b) by stacking the four machine rows, and (c) by using np. arrange as an index map to pick the values out of a flat array and reshape them into the 4 × 5 layout.

Python code

```
flat = [70, 72, 75, 71, 90, 65, 66, 64, 67, 66, 80, 82, 85, 88, 91, 60, 61, 59, 62, 60]

a = np.array(flat).reshape(4, 5)                    # (a) list + reshape
b = np.vstack([[70, 72, 75, 71, 90], [65, 66, 64, 67, 66],
               [80, 82, 85, 88, 91], [60, 61, 59, 62, 60]])   # (b) row stacking
c = np.array(flat)[np.arange(20)].reshape(4, 5)       # (c) arange index map + reshape

print("a == temps :", np.array_equal(a, temps))
print("b == temps :", np.array_equal(b, temps))
print("c == temps :", np.array_equal(c, temps))
print("All close  :", np.allclose(a, b) and np.allclose(b, c))
print("np.arange(20).reshape(4,5) shows the row-major layout:")
print(np.arange(20).reshape(4, 5))
```

Output

```
a == temps: True
b == temps: True
c == temps: True
```

```
All close: True
np.arange(20).reshape(4,5) shows the row-major layout:
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
```

np.array_equal returns True for all three constructions, so the data are identical regardless of how the array was built. The arange layout confirms NumPy's default row-major (C) order: element (i, j) is stored at flat position 5·i + j.

Step 3 – Fahrenheit conversion and day-to-day change (no loops)

The conversion F = C × 9/5 + 32 is applied to the whole array in one vectorised expression. The day-to-day change is obtained by subtracting the array shifted by one day: temps[:, 1:] (Day 2 to Day 5) minus temps[:, :-1] (Day 1 to Day 4).

Python code

```
fahrenheit = temps * 9 / 5 + 32
print("Fahrenheit:\n", fahrenheit)

change_c = temps[:, 1:] - temps[:, :-1]
print("\nDay-to-day change (°C), shape", change_c.shape)
print(change_c)

print("\nCheck with np.diff:", np.array_equal(change_c, np.diff(temps, axis=1)))
print("Change in °F = 1.8 x change in °C:", np.allclose(np.diff(fahrenheit, axis=1), 1.8 * change_c))
```

Output

```
Fahrenheit:
 [[158.  161.6 167.  159.8 194. ]
 [149.  150.8 147.2 152.6 150.8]
 [176.  179.6 185.  190.4 195.8]
 [140.  141.8 138.2 143.6 140. ]]

Day-to-day change (°C), shape (4, 4)
[[ 2  3 -4 19]
```

```
 [ 1 -2  3 -1]
 [ 2  3  3  3]
 [ 1 -2  3 -2]]

Check with np.diff: True
Change in °F = 1.8 x change in °C: True
```

Slicing produces two (4, 4) views that are subtracted element by element. M1 shows the largest jump (+19 °C between Day 4 and Day 5, from 71 to 90), which is a candidate anomaly. A temperature difference scales by 1.8 when converted to °F because the +32 offset cancels in a difference.

Step 4 – Broadcasting: subtract each machine's own mean

Each machine's mean is computed along axis 1 (across days). With keepdims=True the result has shape (4, 1), which can be broadcast against the (4, 5) data.

Python code

```
means = temps.mean(axis=1, keepdims=True)
centred = temps - means
print("temps shape :", temps.shape)
print("means shape :", means.shape)
print("result shape:", centred.shape)
print("Machine means:", means.ravel())
print(np.round(centred, 2))
print("Row sums of centred data (should be ~0):", np.round(centred.sum(axis=1), 10))

# What happens if the means are left as a 1-D array of shape (4,)?
try:
    temps - temps.mean(axis=1)
except ValueError as e:
    print("\nWithout keepdims ->", e)
```

Output

```
temps shape : (4, 5)
means shape : (4, 1)
result shape: (4, 5)
Machine means: [75.6 65.6 85.2 60.4]
```

```
[[-5.6 -3.6 -0.6 -4.6 14.4]
 [-0.6  0.4 -1.6  1.4  0.4]
 [-5.2 -3.2 -0.2  2.8  5.8]
 [-0.4  0.6 -1.4  1.6 -0.4]]
Row sums of centred data (should be ~0): [ 0.  0. -0.  0.]


Without keepdims -> operands could not be broadcast together with shapes (4,5) (4,)
```

Broadcasting compares shapes from the right: (4, 5) and (4, 1). The trailing dimensions 5 and 1 are compatible because a dimension of size 1 is stretched (virtually repeated) to match 5, and the leading dimensions 4 and 4 are equal. The result is (4, 5). Without keepdims the mean has shape (4,), which would be aligned with the last axis (5) and fails because $4 \neq 5$ – so the shape (4, 1) is what makes the operation valid and ensures each row is reduced by its own machine's mean. After centring, every row sums to zero.

Step 5 – Seeded random measurement errors

A seeded Generator (default_rng) produces normally distributed errors (mean 0, standard deviation 0.5 °C). Re-creating the generator with the same seed reproduces the numbers exactly.

Python code

```
rng = np.random.default_rng(seed=42)
errors = rng.normal(loc=0.0, scale=0.5, size=(4, 5))
noisy = temps + errors
print("errors shape:", errors.shape)
print(np.round(errors, 3))
print("\nNoisy readings:\n", np.round(noisy, 2))

rng2 = np.random.default_rng(seed=42)               # same seed again
noisy2 = temps + rng2.normal(0.0, 0.5, size=(4, 5))
print("\nIdentical with same seed     :", np.array_equal(noisy, noisy2))

rng3 = np.random.default_rng(seed=7)                # different seed
noisy3 = temps + rng3.normal(0.0, 0.5, size=(4, 5))
print("Identical with different seed:", np.array_equal(noisy, noisy3))
```

Output

```
errors shape: (4, 5)
[[ 0.152 -0.52   0.375  0.47  -0.976]
 [-0.651  0.064 -0.158 -0.008 -0.427]
 [ 0.44   0.389  0.033  0.564  0.234]
 [-0.43   0.184 -0.479  0.439 -0.025]]

Noisy readings:
 [[70.15 71.48 75.38 71.47 89.02]
 [64.35 66.06 63.84 66.99 65.57]
 [80.44 82.39 85.03 88.56 91.23]
 [59.57 61.18 58.52 62.44 59.98]]

Identical with same seed     : True
Identical with different seed: False
```

The result is identical on re-running with seed 42 and different for seed 7. Fixing the seed makes any experiment involving random numbers reproducible, which is essential for checking and comparing results.

Step 6 – Mathematical functions used meaningfully

Six universal functions are applied, each for a purpose in this temperature-monitoring context.

Python code

```
# np.sqrt : root-mean-square deviation of each machine = population standard deviation
spread = np.sqrt(np.mean(centred ** 2, axis=1))
print("np.sqrt  -> spread per machine :", np.round(spread, 2))

# np.exp : Gaussian 'normality weight' of each reading (1 = at the machine mean)
weight = np.exp(-(centred ** 2) / (2 * spread[:, None] ** 2))
print("np.exp   -> weight of M1 readings:", np.round(weight[0], 3))

# np.log : daily relative (percentage) change = difference of logs
log_change = np.diff(np.log(temps), axis=1) * 100
print("np.log   -> M1 daily % change   :", np.round(log_change[0], 2))
```

```
# np.round : report noisy readings to 1 decimal (instrument resolution)
print("np.round -> noisy M2 readings   :", np.round(noisy[1], 1))

# np.clip : limit readings to the sensor's valid range 62-85 °C
clipped = np.clip(temps, 62, 85)
print("np.clip  ->\n", clipped)

# np.where : flag readings above the 85 °C alarm limit
alarm = np.where(temps > 85, "ALARM", "ok")
print("np.where ->\n", alarm)
print("Alarm positions (machine, day):", [tuple(int(v) + 1 for v in p) for p in np.argwhere(temps > 85)])
```

Output

```
np.sqrt  -> spread per machine : [7.39 1.02 3.97 1.02]
np.exp   -> weight of M1 readings: [0.751 0.888 0.997 0.824 0.15 ]
np.log   -> M1 daily % change   : [ 2.82  4.08 -5.48 23.71]
np.round -> noisy M2 readings   : [64.3 66.1 63.8 67.  65.6]
np.clip  ->
 [[70 72 75 71 85]
 [65 66 64 67 66]
 [80 82 85 85 85]
 [62 62 62 62 62]]
np.where ->
 [['ok' 'ok' 'ok' 'ok' 'ALARM']
 ['ok' 'ok' 'ok' 'ok' 'ok']
 ['ok' 'ok' 'ok' 'ALARM' 'ALARM']
 ['ok' 'ok' 'ok' 'ok' 'ok']]
Alarm positions (machine, day): [(1, 5), (3, 4), (3, 5)]
```

M1 has the highest relative spread because of the Day 5 reading (90 °C, a weight close to 0 under the Gaussian weighting), M3 reaches the alarm threshold on Days 4 and 5 (88 and 91 °C), and the clip function shows how out-of-range values would be limited. The functions work on the whole array at once, so no loops are needed.

Reasoning Inquiry – predicted versus actual shapes

Before running, the result shape of each operation was predicted from the rules of NumPy (reduction removes an axis unless keepdims is used; slicing a range of n columns gives n − 1 differences; element-wise operations keep the shape). The predictions are compared below with the actual shapes.

Python code

```
predictions = {
    "temps (create array)":              ((4, 5), temps.shape),
    "astype(float32)":                   ((4, 5), t32.shape),
    "Fahrenheit (vectorised)":           ((4, 5), fahrenheit.shape),
    "day-to-day change (slicing)":       ((4, 4), change_c.shape),
    "mean(axis=1, keepdims=True)":       ((4, 1), means.shape),
    "temps - means (broadcast)":         ((4, 5), centred.shape),
    "random errors":                     ((4, 5), errors.shape),
    "temps + errors":                    ((4, 5), noisy.shape),
    "np.clip / np.where":                ((4, 5), clipped.shape),
    "np.sqrt(mean(centred**2, axis=1))": ((4,),   spread.shape),
}
print(f"{'Operation':38s}{'Predicted':>12s}{'Actual':>12s}  Match")
for k, (p, a) in predictions.items():
    print(f"{k:38s}{str(p):>12s}{str(a):>12s}  {p == a}")
```

Output

```
Operation                             Predicted      Actual  Match
temps (create array)                     (4, 5)      (4, 5)  True
astype(float32)                          (4, 5)      (4, 5)  True
Fahrenheit (vectorised)                  (4, 5)      (4, 5)  True
day-to-day change (slicing)              (4, 4)      (4, 4)  True
mean(axis=1, keepdims=True)              (4, 1)      (4, 1)  True
temps - means (broadcast)                (4, 5)      (4, 5)  True
random errors                            (4, 5)      (4, 5)  True
temps + errors                           (4, 5)      (4, 5)  True
np.clip / np.where                       (4, 5)      (4, 5)  True
np.sqrt(mean(centred**2, axis=1))           (4,)        (4,)  True
```

Reasoning Inquiry – view versus copy

A view shares the same memory as the original array (basic slicing returns views), so changing the view changes the original. A copy owns its own memory (fancy/boolean indexing and .copy() return copies), so changing it does not affect the original. The difference matters when data is edited or cleaned.

Python code

```
data = temps.copy()                 # work on a copy so 'temps' stays intact

view = data[0, :]                   # basic slice  -> VIEW
copy_ = data[[0], :].copy()         # explicit copy
fancy = data[[0, 2], :]             # fancy index -> COPY

print("view  shares memory with data:", np.shares_memory(view, data))
print("fancy shares memory with data:", np.shares_memory(fancy, data))

view[4] = 75                        # 'correct' the suspicious M1 Day-5 reading in the view
print("After editing the view, data[0] =", data[0], "<- original changed")
fancy[0, 4] = 0                     # editing a copy
print("After editing the fancy copy, data[0, 4] =", data[0, 4], "<- original unchanged")
print("Untouched original temps[0] =", temps[0])
```

Output

```
view  shares memory with data: True
fancy shares memory with data: False
After editing the view, data[0] = [70 72 75 71 75] <- original changed
After editing the fancy copy, data[0, 4] = 75 <- original unchanged
Untouched original temps[0] = [70 72 75 71 90]
```

Example from this code: when the suspicious M1 Day-5 reading (90 °C) was set to 75 through the slice data[0, :], the original array was silently modified because the slice is a view. If the aim had been to keep the raw readings for audit, data[0, :].copy() should have been used. Conversely, the fancy-indexed array data[[0, 2], :] is a copy, so assigning to it does not update the original – a common source of ‘the change did not stick’ bugs.

Q2 – Index, Filter, Summarise and Save Numerical Data

Step 1 – Reproducible 50 × 30 dataset with inserted extreme values

Temperatures of 50 machines over 30 days are drawn from a normal distribution (mean 75 °C, standard deviation 4 °C) with a fixed seed. Four extreme values are then inserted at known positions (0-based row, column) so that the detection step can be checked.

Python code

```
import numpy as np, time

rng = np.random.default_rng(2026)
A = rng.normal(loc=75, scale=4, size=(50, 30))

inserted = {(4, 10): 110.0, (17, 3): 40.0, (33, 25): 115.0, (41, 14): 35.0}
for (r, c), v in inserted.items():
    A[r, c] = v

print("shape:", A.shape, "| dtype:", A.dtype)
print("Inserted extreme values at (row, col):", inserted)
```

Output

```
shape: (50, 30) | dtype: float64
Inserted extreme values at (row, col): {(4, 10): 110.0, (17, 3): 40.0, (33, 25): 115.0, (41, 14): 35.0}
```

Step 2 – Basic slicing, fancy indexing and boolean indexing

Machines are numbered from 1, so “machines 3, 7 and 12” are rows 2, 6 and 11 (0-based). The threshold for boolean indexing is chosen as 90 °C.

Python code

```
last7_first10 = A[:10, -7:]                # basic slicing  -> view
m_3_7_12 = A[[2, 6, 11], :]                # fancy indexing -> copy
threshold = 90
above = A[A > threshold]                   # boolean indexing -> 1-D copy

print("Last 7 days of first 10 machines : shape", last7_first10.shape)
```

```
print("Machines 3, 7, 12 (all days)     : shape", m_3_7_12.shape)
print("Readings above", threshold, "°C         :", np.round(above, 1))
print("Positions of those readings      :", [tuple(map(int, p)) for p in np.argwhere(A > threshold)])
```

Output

```
Last 7 days of first 10 machines : shape (10, 7)
Machines 3, 7, 12 (all days)     : shape (3, 30)
Readings above 90 °C         : [110. 115.]
Positions of those readings      : [(4, 10), (33, 25)]
```

Basic slicing keeps the array structure (10 × 7) and returns a view; fancy indexing with a list of row numbers returns a (3 × 30) copy; the boolean mask returns a flat array of only the readings that satisfy the condition.

Step 3 – Summary statistics: whole array, per machine (axis = 1), per day (axis = 0)

Python code

```
def summary(x, axis=None):
    return dict(mean=np.mean(x, axis=axis), median=np.median(x, axis=axis),
                std=np.std(x, axis=axis), min=np.min(x, axis=axis),
                max=np.max(x, axis=axis), p90=np.percentile(x, 90, axis=axis))

whole = summary(A)
print("WHOLE ARRAY")
for k, v in whole.items():
    print(f"  {k:7s}: {v:.3f}")

per_machine = summary(A, axis=1)     # collapses the 30 days  -> one value per machine
per_day     = summary(A, axis=0)     # collapses the 50 machines -> one value per day
print("\nPER MACHINE (axis=1): result shape", per_machine["mean"].shape)
print("  mean  of machines 1-5 :", np.round(per_machine["mean"][:5], 2))
print("  p90   of machines 1-5 :", np.round(per_machine["p90"][:5], 2))
print("PER DAY (axis=0): result shape", per_day["mean"].shape)
print("  mean  of days 1-5     :", np.round(per_day["mean"][:5], 2))
print("  max   of days 1-5     :", np.round(per_day["max"][:5], 2))
```

Output

```
WHOLE ARRAY
  mean   : 74.927
  median : 74.948
  std    : 4.476
  min    : 35.000
  max    : 115.000
  p90    : 80.094

PER MACHINE (axis=1): result shape (50,)
  mean  of machines 1-5 : [74.99 76.86 74.35 74.58 76.97]
  p90   of machines 1-5 : [78.13 81.7  80.15 79.69 81.56]
PER DAY (axis=0): result shape (30,)
  mean  of days 1-5     : [74.89 74.88 74.91 74.51 75.64]
  max   of days 1-5     : [85.15 87.94 83.97 84.72 82.9 ]
```

The axis argument names the dimension that is collapsed. For the (50, 30) array, axis = 1 collapses the 30 columns (days) and leaves one value for each of the 50 machines, giving a result of shape (50,). axis = 0 collapses the 50 rows (machines) and leaves one value for each of the 30 days, giving shape (30,). With no axis, all 1500 values are reduced to a single number.

Step 4 – Outlier detection with the z-score rule (|z| > 3)

The z-score is z = (x − mean) / std computed over the whole array. np.where returns the row and column positions of readings with |z| > 3, which are compared with the inserted positions.

Python code

```
z = (A - A.mean()) / A.std()
rows, cols = np.where(np.abs(z) > 3)
found = {(int(r), int(c)) for r, c in zip(rows, cols)}
print("Flagged positions (row, col):", sorted(found))
print("Their values               :", np.round(A[rows, cols], 1))
print("Their z-scores             :", np.round(z[rows, cols], 2))
print("Inserted positions all found:", set(inserted) <= found)
print("Extra (naturally extreme)   :", sorted(found - set(inserted)))
```

Output

```
Flagged positions (row, col): [(4, 10), (6, 8), (17, 3), (33, 25), (41, 14)]
Their values                : [110.  60.  40. 115.  35.]
Their z-scores              : [ 7.84 -3.33 -7.8   8.95 -8.92]
Inserted positions all found: True
Extra (naturally extreme)   : [(6, 8)]
```

All four inserted extreme values are detected (z-scores of about +7.8, −7.8, +9.0 and −8.9). One additional reading, row 6 / column 8 (60.0 °C, z = −3.33), is also flagged. It was not inserted; it is a genuine tail value of the normal distribution – about 0.27 % of 1,500 readings (≈ 4) are expected beyond 3 standard deviations by chance – so the rule has one ‘false alarm’ and is reported honestly rather than hidden. The four inserted positions are matched exactly.

Step 5 – Replace unusual values by the machine median using masks (no loop)

Python code

```
mask = np.abs(z) > 3
machine_median = np.median(A, axis=1, keepdims=True)          # shape (50, 1)
clean = np.where(mask, machine_median, A)                     # broadcast (50,1) -> (50,30)


print("Mask shape / True count :", mask.shape, "/", int(mask.sum()))
print("Shape unchanged         :", clean.shape == A.shape)
print("Cells changed           :", int((clean != A).sum()))
for (r, c) in sorted(found):
    print(f"  machine row {r:2d}, day col {c:2d}: {A[r, c]:7.2f} -> {clean[r, c]:6.2f}")
print("Inserted original array unchanged:", np.isclose(A[4, 10], 110.0))
```

Output

```
Mask shape / True count : (50, 30) / 5
Shape unchanged         : True
Cells changed           : 5
  machine row  4, day col 10:  110.00 ->  75.86
  machine row  6, day col  8:   60.01 ->  75.92
  machine row 17, day col  3:   40.00 ->  76.01
  machine row 33, day col 25:  115.00 ->  75.45
```

```
 machine row 41, day col 14:   35.00 ->  75.01
Inserted original array unchanged: True
```

np.where(mask, replacement, original) selects the machine median where the mask is True and the original value otherwise. The median of the same machine is used because it is itself not distorted by the outlier. The shape (50, 30) is unchanged and only the flagged cells are modified.

Step 6 – Which statistic is most / least affected by the unusual readings?

Python code

```
before = summary(A); after = summary(clean)
print(f"{'statistic':8s}{'with outliers':>15s}{'cleaned':>12s}{'abs change':>12s}{'% change':>11s}")
for k in before:
    b, a = before[k], after[k]
    print(f"{k:8s}{b:15.3f}{a:12.3f}{abs(a-b):12.3f}{100*abs(a-b)/abs(b):10.2f}%")
```

Output

```
statistic  with outliers     cleaned  abs change   % change
mean            74.927      74.939       0.012      0.02%
median          74.948      75.002       0.054      0.07%
std             4.476       4.016        0.461     10.29%
min             35.000      62.554      27.554     78.73%
max            115.000      88.334      26.666     23.19%
p90             80.094      80.067       0.027      0.03%
```

Step 7 – Loop-based versus vectorised per-machine mean (150,000 elements)

Python code

```
big = np.random.default_rng(1).normal(75, 4, size=(500, 300))   # 150,000 elements
print("Elements:", big.size)


def mean_loop(x):
    out = []
    for i in range(x.shape[0]):
        s = 0.0
        for j in range(x.shape[1]):
```

```
            s += x[i, j]
        out.append(s / x.shape[1])
    return np.array(out)

t0 = time.perf_counter(); m_loop = mean_loop(big);       t_loop = time.perf_counter() - t0
t0 = time.perf_counter(); m_vec  = big.mean(axis=1);     t_vec  = time.perf_counter() - t0

print(f"Loop-based : {t_loop*1000:9.2f} ms")
print(f"Vectorised : {t_vec*1000:9.3f} ms")
print(f"Speed-up   : {t_loop/t_vec:9.0f} x")
print("Same result:", np.allclose(m_loop, m_vec))
```

Output

```
Elements: 150000
Loop-based :     14.59 ms
Vectorised :     0.157 ms
Speed-up   :        93 x
Same result: True
```

The vectorised call is dozens of times faster (about 60× in this run) because the loop runs in the Python interpreter (one object creation and type check per element), whereas big.mean(axis=1) runs in compiled C over contiguous memory. Exact timings vary from run to run, but the ratio is always large.

Step 8 – Save with np.save, np.savez and np.savetxt and verify the round trip

Python code

```
np.save("clean.npy", clean)
np.savez("clean.npz", clean=clean, machine_median=machine_median)
np.savetxt("clean.txt", clean, delimiter=",")              # default fmt = %.18e

r1 = np.load("clean.npy")
r2 = np.load("clean.npz")["clean"]
r3 = np.loadtxt("clean.txt", delimiter=",")

for name, r in [("np.save  ", r1), ("np.savez ", r2), ("np.savetxt", r3)]:
```

```
    print(f"{name}: equal={np.array_equal(r, clean)}  dtype={r.dtype}  shape={r.shape}")


# When is savetxt a poor choice?  Use a typed (int32) array and a 2-decimal format.
small = np.array([[1, 2], [3, 4]], dtype=np.int32)
np.save("small.npy", small); np.savetxt("small.txt", small, delimiter=",", fmt="%d")
print("\nint32 array  -> np.load dtype:", np.load("small.npy").dtype,
      "| np.loadtxt dtype:", np.loadtxt("small.txt", delimiter=",").dtype)
np.savetxt("clean_2dp.txt", clean, delimiter=",", fmt="%.2f")
lossy = np.loadtxt("clean_2dp.txt", delimiter=",")
print("savetxt with fmt='%.2f' exact?:", np.array_equal(lossy, clean),
      "| max error:", float(np.abs(lossy - clean).max()))
import os
print("File sizes (bytes): npy =", os.path.getsize("clean.npy"),
      "| npz =", os.path.getsize("clean.npz"), "| txt =", os.path.getsize("clean.txt"))
```

Output

```
np.save  : equal=True  dtype=float64  shape=(50, 30)
np.savez : equal=True  dtype=float64  shape=(50, 30)
np.savetxt: equal=True  dtype=float64  shape=(50, 30)

int32 array  -> np.load dtype: int32 | np.loadtxt dtype: float64
savetxt with fmt='%.2f' exact?: False | max error: 0.0049989413748932066
File sizes (bytes): npy = 12128 | npz = 12924 | txt = 37500
```

All three formats return exactly the same values for the float64 array. np.save and np.savez keep the dtype, shape and full precision automatically (np.savez can also store several arrays by name). np.savetxt is a poor choice for large arrays, for typed data (an int32 array comes back as float64 unless the dtype is specified again), when a short format (e.g. %.2f) is used because precision is lost, for arrays with more than two dimensions, and when speed or file size matters – the text file is several times larger and much slower to read and write. It is only preferable when a human-readable file or exchange with other software (e.g. Excel) is required.

Analysis Inquiry – answer supported by the numbers

Python code

```
rel = {k: abs(after[k] - before[k]) / abs(before[k]) * 100 for k in before}
ranked = sorted(rel.items(), key=lambda kv: -kv[1])
print("Statistics ranked from most to least affected (% change):")
for k, v in ranked:
    print(f"  {k:7s} {v:8.3f} %")
```

Output

```
Statistics ranked from most to least affected (% change):
  min       78.727 %
  max       23.188 %
  std       10.293 %
  median     0.072 %
  p90        0.034 %
  mean       0.016 %
```

The minimum is the most affected statistic (it changes by 78.7 %, from 35.0 to 62.6 °C), followed by the maximum (23.2 %, from 115.0 to 88.3 °C) and the standard deviation (10.3 %, from 4.48 to 4.02), which is inflated because deviations are squared. The least affected is the mean (0.016 %), followed by the 90th percentile (0.034 %) and the median (0.072 %). The mean is stable here only because the two high outliers (110, 115) and the two low outliers (40, 35) happen to cancel each other; with one-sided outliers it would move much more. The median and percentiles are robust by construction because they depend on rank rather than on the size of extreme values. This is why the machine median was chosen as the replacement value.

Q3 – Load, Inspect and Clean Data Using Pandas DataFrames

Step 1 – Write the sample to CSV and read it with pd.read_csv

The sample is written exactly as given (the missing Defects value is an empty field, the “N/A” is text, and the last row has a lower-case line code). The file is read twice: first with all automatic missing-value recognition switched off (keep_default_na=False) to show the problem, then with na_values=['N/A'] and parse_dates=['Date'].

Python code

```
import pandas as pd, numpy as np, io

sample_csv = """Machine_ID,Date,Line,Output_Units,Defects
M01,2026-03-01,A,480,12
M02,2026-03-01,A,455,9
M03,2026-03-01,B,N/A,15
M01,2026-03-02,A,492,
M02,2026-03-02,A,455,9
M02,2026-03-02,A,455,9
M03,2026-03-02,b,510,14
"""
with open("sample.csv", "w") as f:
    f.write(sample_csv)

naive = pd.read_csv("sample.csv", keep_default_na=False)   # no automatic NA recognition
print("Without NA handling -> Output_Units dtype:", naive["Output_Units"].dtype,
      "| values:", naive["Output_Units"].tolist())

df = pd.read_csv("sample.csv", na_values=["N/A"], parse_dates=["Date"])
print("With na_values=['N/A'] -> Output_Units dtype:", df["Output_Units"].dtype,
      "| Date dtype:", df["Date"].dtype)
print(df)
```

Output

```
Without NA handling -> Output_Units dtype: str | values: ['480', '455', 'N/A', '492', '455', '455', '510']
With na_values=['N/A'] -> Output_Units dtype: float64 | Date dtype: datetime64[us]
```

```
  Machine_ID       Date Line  Output_Units  Defects
0       M01 2026-03-01    A         480.0     12.0
1       M02 2026-03-01    A         455.0      9.0
2       M03 2026-03-01    B           NaN     15.0
3       M01 2026-03-02    A         492.0      NaN
4       M02 2026-03-02    A         455.0      9.0
5       M02 2026-03-02    A         455.0      9.0
6       M03 2026-03-02    b         510.0     14.0
```

With automatic recognition switched off, the single text entry “N/A” makes pandas read Output_Units as text (str), so no arithmetic is possible. na_values=['N/A'] declares which strings mean “missing”, converts them to NaN and the column becomes numeric (float64, because NaN cannot be stored in a plain integer column). Note that pandas already treats “N/A” as missing by default; stating na_values explicitly documents the rule and is essential for other markers such as “-”, “missing” or “?”. parse_dates turns Date into datetime64, which allows date sorting, filtering and resampling.

Step 2 – Inspect the DataFrame

Python code

```
print("--- head() ---");  print(df.head(3))
print("\n--- info() ---"); df.info()
print("\n--- describe() ---"); print(df.describe())
print("\n--- dtypes ---");  print(df.dtypes)
print("\n--- shape ---");   print(df.shape)
print("\n--- isna().sum() ---"); print(df.isna().sum())
print("\nDuplicated rows:", int(df.duplicated().sum()))
print("Line values:", df["Line"].unique().tolist())
```

Output

```
--- head() ---
  Machine_ID       Date Line  Output_Units  Defects
0       M01 2026-03-01    A         480.0     12.0
1       M02 2026-03-01    A         455.0      9.0
2       M03 2026-03-01    B           NaN     15.0
```

```
--- info() ---
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Machine_ID    7 non-null      str
 1   Date          7 non-null      datetime64[us]
 2   Line          7 non-null      str
 3   Output_Units  6 non-null      float64
 4   Defects       6 non-null      float64
dtypes: datetime64[us](1), float64(2), str(2)
memory usage: 412.0 bytes

--- describe() ---
                      Date  Output_Units    Defects
count                    7      6.000000   6.000000
mean   2026-03-01 13:42:51.428571    474.500000  11.333333
min          2026-03-01 00:00:00    455.000000   9.000000
25%          2026-03-01 00:00:00    455.000000   9.000000
50%          2026-03-02 00:00:00    467.500000  10.500000
75%          2026-03-02 00:00:00    489.000000  13.500000
max          2026-03-02 00:00:00    510.000000  15.000000
std                          NaN     23.398718   2.732520

--- dtypes ---
Machine_ID              str
Date           datetime64[us]
Line                    str
Output_Units          float64
Defects               float64
dtype: object

--- shape ---
(7, 5)
```

```
--- isna().sum() ---
Machine_ID      0
Date            0
Line            0
Output_Units    1
Defects         1
dtype: int64

Duplicated rows: 1
Line values: ['A', 'B', 'b']
```

head() shows the first rows and the layout of the data. info() shows 7 rows and the non-null count of each column: Output_Units has 6 and Defects has 6, so each is missing one value; it also reveals the data types. describe() gives count, mean, spread and range of the numeric columns – the count (6 instead of 7) again reveals missing data, and unrealistic minimum/maximum values would reveal errors. dtypes confirms that Date is datetime64 and Output_Units/Defects are float64 (NaN present). shape gives (7, 5). isna().sum() counts one missing value in each of Output_Units and Defects. Two further problems are not visible in those summaries but are found by duplicated() and unique(): one exact duplicate row and an inconsistent Line code (‘b’ versus ‘B’).

Step 3 – Clean the data with a logged count for each step

Cleaning order: remove exact duplicates → standardise Line → convert Output_Units to numeric → handle missing values. Two strategies for missing values are compared: dropping rows and filling values.

Python code

```
log = []
c = df.copy()

n0 = len(c); c = c.drop_duplicates()
log.append(("Remove exact duplicate rows", n0 - len(c), "rows removed"))

before = c["Line"].tolist()
c["Line"] = c["Line"].str.strip().str.upper()
log.append(("Standardise Line (strip + upper)", sum(b != a for b, a in zip(before, c["Line"])),
```

```
"values changed"))

na_before = int(c["Output_Units"].isna().sum())
c["Output_Units"] = pd.to_numeric(c["Output_Units"], errors="coerce")
log.append(("Convert Output_Units to numeric", int(c["Output_Units"].isna().sum()) - na_before,
"new NaN created"))

# Strategy 1: drop rows with missing values
dropped = c.dropna(subset=["Output_Units", "Defects"])
# Strategy 2: fill values (Output with the machine's median, Defects with 0 or median)
filled = c.copy()
filled["Output_Units"] =
filled["Output_Units"].fillna(filled.groupby("Machine_ID")["Output_Units"].transform("median"))
filled["Defects"] =
filled["Defects"].fillna(filled.groupby("Machine_ID")["Defects"].transform("median"))
print("Rows after duplicates removed:", len(c))
print("Strategy 1 (drop)  keeps", len(dropped), "rows")
print("Strategy 2 (fill)  keeps", len(filled), "rows; still missing:", int(filled.isna().sum().sum()))
print(filled)

# CHOSEN strategy: drop rows with missing Output_Units; fill missing Defects with a rate-based
estimate
clean = c.dropna(subset=["Output_Units"]).copy()
rate_by_machine = (clean["Defects"] /
clean["Output_Units"]).groupby(clean["Machine_ID"]).median()
miss = clean["Defects"].isna()
clean.loc[miss, "Defects"] = (clean.loc[miss, "Machine_ID"].map(rate_by_machine) *
clean.loc[miss, "Output_Units"]).round()
log.append(("Drop rows with missing Output_Units", len(c) - len(clean), "rows removed"))
log.append(("Fill missing Defects (machine median rate x output)", int(miss.sum()), "values
filled"))
clean = clean.astype({"Output_Units": "int64", "Defects": "int64"})

print("\nCLEANING LOG")
for step, n, what in log:
```

```
    print(f"  {step:55s}{n:3d} {what}")
print("\nCleaned sample:"); print(clean)
```

Output

```
Rows after duplicates removed: 6
Strategy 1 (drop)  keeps 4 rows
Strategy 2 (fill)  keeps 6 rows; still missing: 0
  Machine_ID       Date Line  Output_Units  Defects
0        M01 2026-03-01    A         480.0     12.0
1        M02 2026-03-01    A         455.0      9.0
2        M03 2026-03-01    B         510.0     15.0
3        M01 2026-03-02    A         492.0     12.0
4        M02 2026-03-02    A         455.0      9.0
6        M03 2026-03-02    B         510.0     14.0

CLEANING LOG
  Remove exact duplicate rows                          1 rows removed
  Standardise Line (strip + upper)                     1 values changed
  Convert Output_Units to numeric                      0 new NaN created
  Drop rows with missing Output_Units                  1 rows removed
  Fill missing Defects (machine median rate x output)      1 values filled

Cleaned sample:
  Machine_ID       Date Line  Output_Units  Defects
0        M01 2026-03-01    A          480       12
1        M02 2026-03-01    A          455        9
3        M01 2026-03-02    A          492       12
4        M02 2026-03-02    A          455        9
6        M03 2026-03-02    B          510       14
```

Justification. Output_Units: the row is dropped. Output is the measured quantity itself; inventing it would invent production, and without it Defect_Rate cannot be calculated. Defects: the value is filled. The row has valid output, so dropping it would lose good information; defects scale with output, so the machine's median defect rate multiplied by that day's output is a defensible estimate (here M01's rate of 12/480 = 2.5 % gives 492 × 0.025 ≈ 12). In general dropping is appropriate when the missing quantity is the main measurement or the data is plentiful, and filling is appropriate when

the value can be estimated from related data and the row has other valuable information. Filled values must be flagged and their impact checked.

Step 4 – Selection with columns, .loc, .iloc and boolean conditions

Python code

```
print("Column selection (one):\n", clean["Output_Units"].tolist())
print("\nColumn selection (several):\n", clean[["Machine_ID", "Defects"]])

print("\n.loc  (label-based) rows labelled 0..2, two columns:\n",
     clean.loc[0:2, ["Machine_ID", "Output_Units"]])         # end label INCLUDED
print("\n.iloc (position-based) first 2 rows, columns 0 and 3:\n",
     clean.iloc[0:2, [0, 3]])                               # end position EXCLUDED

cond = (clean["Line"] == "A") & (clean["Output_Units"] > 470)    # two columns combined
print("\nBoolean: Line == 'A' and Output_Units > 470:\n", clean.loc[cond])

# Chained indexing: unreliable
tmp = clean.copy()
with __import__("warnings").catch_warnings(record=True) as w:
    __import__("warnings").simplefilter("always")
    tmp[tmp["Line"] == "A"]["Defects"] = 0           # chained: row filter then column assignment
    print("\nChained assignment warning:", w[0].category.__name__ if w else "none")
print("After chained assignment, Line A defects:", tmp.loc[tmp["Line"] == "A", "Defects"].tolist(),
"<- unchanged")
tmp.loc[tmp["Line"] == "A", "Defects"] = 0           # single .loc call
print("After .loc assignment,     Line A defects:", tmp.loc[tmp["Line"] == "A", "Defects"].tolist())
```

Output

```
Column selection (one):
 [480, 455, 492, 455, 510]

Column selection (several):
   Machine_ID  Defects
0        M01       12
1        M02        9
```

```
3       M01       12
4       M02        9
6       M03       14

.loc  (label-based) rows labelled 0..2, two columns:
  Machine_ID  Output_Units
0       M01          480
1       M02          455

.iloc (position-based) first 2 rows, columns 0 and 3:
  Machine_ID  Output_Units
0       M01          480
1       M02          455

Boolean: Line == 'A' and Output_Units > 470:
  Machine_ID       Date Line  Output_Units  Defects
0       M01 2026-03-01    A          480       12
3       M01 2026-03-02    A          492       12

Chained assignment warning: ChainedAssignmentError
After chained assignment, Line A defects: [12, 9, 12, 9] <- unchanged
After .loc assignment,     Line A defects: [0, 0, 0, 0]
```

The boolean condition uses & with brackets around each comparison and returns the Line A rows with more than 470 units. .loc uses labels and includes the end label; .iloc uses integer positions and excludes the end position. Chained indexing, tmp[mask]['Defects'] = 0, performs two separate operations: the first (tmp[mask]) returns a temporary object that may be a copy, and the assignment is applied to that temporary object, so the original DataFrame may not change (here it did not). Whether it works depends on internal details, and recent pandas versions raise a warning or never update the original (copy-on-write). A single call, tmp.loc[mask, 'Defects'] = 0, is unambiguous and reliable.

Step 5 – Derived column and a meaningful index

Python code

```
clean["Defect_Rate"] = clean["Defects"] / clean["Output_Units"]
indexed = clean.set_index(["Machine_ID", "Date"]).sort_index()
print(indexed)
print("\nIndex unique:", indexed.index.is_unique, "| index names:", indexed.index.names)
print("M02 on 2026-03-02:", indexed.loc[("M02", "2026-03-02")].to_dict())
```

Output

```
                  Line  Output_Units  Defects  Defect_Rate
Machine_ID Date
M01        2026-03-01    A          480       12     0.025000
         2026-03-02    A          492       12     0.024390
M02        2026-03-01    A          455        9     0.019780
         2026-03-02    A          455        9     0.019780
M03        2026-03-02    B          510       14     0.027451


Index unique: True | index names: ['Machine_ID', 'Date']
M02 on 2026-03-02: {'Line': 'A', 'Output_Units': 455, 'Defects': 9, 'Defect_Rate':
0.01978021978021978}
```

Defect_Rate is a vectorised division (defects ÷ output). A MultiIndex of Machine_ID and Date identifies each record uniquely (confirmed after duplicate removal) and allows direct label selection such as indexed.loc[('M02', '2026-03-02')].

Step 6 – Reproducible synthetic dataset of 1,000 production records

The dataset has the same columns plus Shift. It is generated with seed 7 from 11 machines on three lines. It deliberately includes: missing Output_Units (more frequent on Night shift) and Defects values, inconsistent Line text, 15 exact duplicate rows, and one record from a machine (M99) that is missing from Machine_Info, which will be needed in Q4. The file is saved with to_csv and used in Q4–Q7.

Python code

```
rng = np.random.default_rng(7)
line_of = {f"M{i:02d}": ("A" if i <= 4 else "B" if i <= 8 else "C") for i in range(1, 12)}
base_rate = dict(zip(line_of, rng.uniform(0.015, 0.045, size=11)))      # machine-specific defect rate
shift_mult = {"Morning": 1.0, "Evening": 1.1, "Night": 1.35}
```

```
dates = pd.date_range("2026-03-01", "2026-04-30")

grid = pd.MultiIndex.from_product([list(line_of), dates, list(shift_mult)],
                                  names=["Machine_ID", "Date", "Shift"]).to_frame(index=False)
grid = grid.sample(n=984, random_state=7).sort_values(["Date", "Machine_ID",
"Shift"]).reset_index(drop=True)
grid["Line"] = grid["Machine_ID"].map(line_of)
grid["Output_Units"] = rng.normal(480, 30, len(grid)).round().astype(int)
rate = grid["Machine_ID"].map(base_rate) * grid["Shift"].map(shift_mult)
grid["Defects"] = rng.binomial(grid["Output_Units"], rate)

# one record whose machine is NOT in Machine_Info (needed in Q4)
extra = pd.DataFrame([{"Machine_ID": "M99", "Date": pd.Timestamp("2026-04-15"), "Shift":
"Night",
                       "Line": "C", "Output_Units": 400, "Defects": 48}])
syn = pd.concat([grid, extra], ignore_index=True)

# deliberately missing values (Output_Units missing more often on Night shift) and Line-text noise
p_out = np.array(syn["Shift"].map({"Morning": 0.015, "Evening": 0.02, "Night": 0.06}),
dtype=float)
p_out[syn["Machine_ID"].eq("M99")] = 0
syn["Output_Units"] = syn["Output_Units"].astype("Int64").mask(rng.random(len(syn)) < p_out)
syn["Defects"] = syn["Defects"].astype("Int64").mask((rng.random(len(syn)) < 0.025) &
(syn["Machine_ID"] != "M99"))
noise = rng.random(len(syn)) < 0.03
syn.loc[noise, "Line"] = syn.loc[noise, "Line"].str.lower()
noise2 = rng.random(len(syn)) < 0.02
syn.loc[noise2, "Line"] = " " + syn.loc[noise2, "Line"]

dups = syn.sample(n=15, random_state=1)                    # exact duplicate rows
raw = pd.concat([syn, dups]).sample(frac=1, random_state=3).reset_index(drop=True)
raw = raw[["Machine_ID", "Date", "Line", "Shift", "Output_Units", "Defects"]]
raw.to_csv("production_raw.csv", index=False)

print("Rows:", len(raw), "| columns:", raw.columns.tolist())
```

```
print(raw.head(5))
print("\nMissing values:\n", raw.isna().sum().to_string())
print("Exact duplicate rows:", int(raw.duplicated().sum()))
print("Line spellings:", sorted(raw["Line"].unique().tolist()))
print("Missing Output_Units rate by Shift (%):")
print((raw.groupby("Shift")["Output_Units"].apply(lambda s: s.isna().mean() *
100)).round(1).to_string())
```

Output

```
Rows: 1000 | columns: ['Machine_ID', 'Date', 'Line', 'Shift', 'Output_Units', 'Defects']
  Machine_ID       Date Line    Shift  Output_Units  Defects
0        M09 2026-04-10    c  Morning           486       16
1        M06 2026-04-17    B    Night           421       25
2        M06 2026-04-26    B  Evening           507       20
3        M09 2026-03-12    C    Night           480       35
4        M08 2026-04-06    B  Morning           471       17

Missing values:
 Machine_ID       0
Date             0
Line             0
Shift            0
Output_Units    37
Defects         25
Exact duplicate rows: 15
Line spellings: [' A', ' B', ' C', ' c', 'A', 'B', 'C', 'a', 'b', 'c']
Missing Output_Units rate by Shift (%):
Shift
Evening    0.6
Morning    1.2
Night      9.5
```

The saved file production_raw.csv has exactly 1,000 rows with the intended problems. Missing output is deliberately more common on the Night shift so that the bias question below has a concrete answer.

Step 7 – Clean the synthetic dataset and log every step

Python code

```
raw = pd.read_csv("production_raw.csv", parse_dates=["Date"],
                  dtype={"Output_Units": "Int64", "Defects": "Int64"})
log = [("Raw records read", len(raw), "")]
d = raw.copy()

n = len(d); d = d.drop_duplicates()
log.append(("1. Remove exact duplicate rows", n - len(d), "rows removed"))

changed = int((d["Line"] != d["Line"].str.strip().str.upper()).sum())
d["Line"] = d["Line"].str.strip().str.upper()
log.append(("2. Standardise Line (strip + upper)", changed, "values changed"))

dup_keys = int(d.duplicated(["Machine_ID", "Date", "Shift"]).sum())
log.append(("3. Duplicate (Machine, Date, Shift) keys after cleaning", dup_keys, "found"))

n = len(d); d = d.dropna(subset=["Output_Units"])
log.append(("4. Drop rows with missing Output_Units", n - len(d), "rows removed"))

d["Defect_Rate_tmp"] = d["Defects"] / d["Output_Units"]
med_rate = d.groupby("Machine_ID")["Defect_Rate_tmp"].transform("median")
miss = d["Defects"].isna()
d["Defects_Imputed"] = miss
d.loc[miss, "Defects"] = (med_rate[miss] * d.loc[miss, "Output_Units"]).round().astype("Int64")
log.append(("5. Fill missing Defects (machine median rate x output)", int(miss.sum()), "values
filled"))

d["Output_Units"] = d["Output_Units"].astype("int64"); d["Defects"] =
d["Defects"].astype("int64")
d["Defect_Rate"] = d["Defects"] / d["Output_Units"]
d = d.drop(columns="Defect_Rate_tmp").reset_index(drop=True)
log.append(("Final clean records", len(d), ""))
d.to_csv("production_clean.csv", index=False)
```

```
for s, n, w in log: print(f"  {s:58s}{n:6d} {w}")
print("\nShift mix before vs after dropping missing Output (%):")
mix =
pd.concat([raw.drop_duplicates()["Shift"].value_counts(normalize=True).mul(100).round(1).renam
e("before"),
           d["Shift"].value_counts(normalize=True).mul(100).round(1).rename("after")], axis=1)
print(mix.to_string())
print("Night-shift share of dropped rows:",
    round(100 * (raw.drop_duplicates().loc[raw.drop_duplicates()["Output_Units"].isna(), "Shift"]
== "Night").mean(), 1), "%")
```

Output

```
  Raw records read                                    1000
  1. Remove exact duplicate rows                       15 rows removed
  2. Standardise Line (strip + upper)                  57 values changed
  3. Duplicate (Machine, Date, Shift) keys after cleaning    0 found
  4. Drop rows with missing Output_Units               37 rows removed
  5. Fill missing Defects (machine median rate x output)     25 values filled
  Final clean records                                 948

Shift mix before vs after dropping missing Output (%):
         before  after
Shift
Morning    33.9   34.8
Evening    33.3   34.4
Night      32.8   30.8
Night-shift share of dropped rows: 83.8 %
```

Each step is justified: duplicates are exact copies and would double-count output; Line codes ‘a’, ‘ a’ and ‘A’ are the same line and would otherwise create false groups; the key check confirms that each machine/date/shift appears once; rows with missing Output_Units are dropped because output cannot be recovered and Defect_Rate would be undefined; missing Defects are estimated from the machine's own defect rate and flagged in Defects_Imputed.

Which step could introduce bias? Step 4 (dropping rows with missing Output_Units). Missingness is not random here – it is more frequent on the Night shift, which also has the highest defect rate – so dropping those rows removes Night-shift records disproportionately and can make the overall defect

rate look better than it is. It can be detected by comparing the Shift (and Machine) distribution before and after the drop (the table above shows Night's share falling), by comparing the mean defect rate of the records kept with that of the complete records of the affected group, and by testing whether the missingness indicator is related to other columns (e.g. missing rate by shift). Step 5 may also bias by shrinking variance towards the median rate, which is why imputed rows are flagged.

Step 8 – Read the saved file back: values, data types and dates preserved

Python code

```
back = pd.read_csv("production_clean.csv", parse_dates=["Date"])
print("Same shape       :", back.shape == d.shape)
print("Same dtypes      :", back.dtypes.equals(d.dtypes))
print("Date dtype       :", back["Date"].dtype, "| min", back["Date"].min().date(), "| max",
back["Date"].max().date())
try:
    pd.testing.assert_frame_equal(back, d, check_dtype=True)
    print("assert_frame_equal: PASSED - values, dtypes and dates are identical")
except AssertionError as e:
    print("assert_frame_equal: FAILED ->", str(e)[:300])

# Without parse_dates the dates come back as text
print("Without parse_dates the Date dtype is:", pd.read_csv("production_clean.csv")["Date"].dtype)

# One other format: JSON (and Excel)
d.to_json("production_clean.json", orient="records", date_format="iso", indent=1)
js = pd.read_json("production_clean.json", orient="records", convert_dates=["Date"])
print("\nJSON round trip equal values:", np.array_equal(js[["Output_Units",
"Defects"]].to_numpy(), d[["Output_Units", "Defects"]].to_numpy()),
      "| dates equal:", bool((js["Date"].dt.date == d["Date"].dt.date).all()))
d.to_excel("production_clean.xlsx", index=False)
xl = pd.read_excel("production_clean.xlsx")
print("Excel round trip equal values:", np.allclose(xl["Defect_Rate"], d["Defect_Rate"]), "| rows:",
len(xl))
```

Output

```
Same shape       : True
Same dtypes      : True
```

```
Date dtype       : datetime64[us] | min 2026-03-01 | max 2026-04-30
assert_frame_equal: PASSED - values, dtypes and dates are identical
Without parse_dates the Date dtype is: str

JSON round trip equal values: True | dates equal: True
Excel round trip equal values: True | rows: 948
```

The CSV round trip reproduces identical values, integer types, boolean flag, string columns and datetime64 dates – but only because parse_dates was supplied: CSV stores no type information, so without it dates return as plain text. JSON and Excel were also written and read back successfully. JSON keeps types a little better for numbers but still stores dates as text (ISO strings); Excel keeps date types but is slower and limited in size.

Q4 – Combine, Group, Filter and Sort Data to Answer Questions

Step 1 – Build Machine_Info and merge: inner, left and outer

Machine_Info has 12 machines (M01–M12, seed 11). M12 has no production records, and the production data contains one record from machine M99 which is not in Machine_Info. The cleaned production file from Q3 is used.

Python code

```
prod = pd.read_csv("production_clean.csv", parse_dates=["Date"])
r = np.random.default_rng(11)
info = pd.DataFrame({
    "Machine_ID": [f"M{i:02d}" for i in range(1, 13)],
    "Line": ["A"] * 4 + ["B"] * 4 + ["C"] * 4,
    "Installed_Year": r.integers(2015, 2025, 12),
    "Supervisor": r.choice(["Anand", "Bhavna", "Chitra"], 12)})
info.to_csv("machine_info.csv", index=False)
print(info.to_string(index=False))

print("\nProduction rows:", len(prod), "| distinct machines:", prod["Machine_ID"].nunique())
print("Machines in production but not in info:", sorted(set(prod.Machine_ID) -
set(info.Machine_ID)))
print("Machines in info but not in production:", sorted(set(info.Machine_ID) -
set(prod.Machine_ID)))

kw = dict(on="Machine_ID", suffixes=("", "_info"), validate="m:1", indicator=True)
inner = prod.merge(info, how="inner", **kw)
left  = prod.merge(info, how="left",  **kw)
outer = prod.merge(info, how="outer", **kw)

print("\nRows -> production:", len(prod), "| inner:", len(inner), "| left:", len(left), "| outer:",
len(outer))
for name, m in [("inner", inner), ("left", left), ("outer", outer)]:
    print(f"\n_merge counts ({name}):"); print(m["_merge"].value_counts().to_string())
print("\nUnmatched production record(s):")
print(left.loc[left["_merge"] == "left_only", ["Machine_ID", "Date", "Shift", "Output_Units",
```

```
"Defects", "Supervisor"]])
print("Unmatched info row(s):")
print(outer.loc[outer["_merge"] == "right_only", ["Machine_ID", "Line_info", "Installed_Year",
"Supervisor", "Output_Units"]])
print("\nLine in production equals Line in Machine_Info for matched rows:",
    bool((inner["Line"] == inner["Line_info"]).all()))
```

Output

```
Machine_ID Line  Installed_Year Supervisor
     M01    A           2016     Bhavna
     M02    A           2016      Anand
     M03    A           2022     Bhavna
     M04    A           2019      Anand
     M05    B           2020     Chitra
     M06    B           2021     Chitra
     M07    B           2022     Chitra
     M08    B           2015     Bhavna
     M09    C           2019     Chitra
     M10    C           2016     Bhavna
     M11    C           2019      Anand
     M12    C           2024     Bhavna


Production rows: 948 | distinct machines: 12
Machines in production but not in info: ['M99']
Machines in info but not in production: ['M12']


Rows -> production: 948 | inner: 947 | left: 948 | outer: 949


_merge counts (inner):
_merge
both          947
left_only       0
right_only      0


_merge counts (left):
_merge
```

```
both         947
left_only      1
right_only     0

_merge counts (outer):
_merge
both         947
left_only      1
right_only     1

Unmatched production record(s):
   Machine_ID       Date  Shift  Output_Units  Defects Supervisor
220        M99 2026-04-15  Night          400       48        NaN
Unmatched info row(s):
   Machine_ID Line_info  Installed_Year Supervisor  Output_Units
947        M12         C         2024.0     Bhavna          NaN

Line in production equals Line in Machine_Info for matched rows: True
```

Explanation of the differences. Inner keeps only keys present in both tables: all production rows except the single M99 record (947 = 948 − 1); machine M12 is dropped because it has no production. Left keeps every production row (948), so the M99 record is retained with NaN in the Machine_Info columns (_merge = left_only). Outer keeps everything: 948 production rows plus one extra row for M12, which has Machine_Info columns but NaN production values (_merge = right_only), giving 949 rows. validate='m:1' guarantees that Machine_Info has unique keys, so the merge cannot silently multiply production rows, and the suffix option keeps the two Line columns apart so that they can be compared.

Step 2 – Split into two monthly files, recombine with pd.concat

Python code

```
march = prod[prod["Date"].dt.month == 3]
april = prod[prod["Date"].dt.month == 4]
march.to_csv("prod_2026_03.csv", index=False); april.to_csv("prod_2026_04.csv", index=False)
m = pd.read_csv("prod_2026_03.csv", parse_dates=["Date"])
```

```
a = pd.read_csv("prod_2026_04.csv", parse_dates=["Date"])
print("March rows:", len(m), "| April rows:", len(a), "| sum:", len(m) + len(a), "| original:",
len(prod))

# Index handling
plain = pd.concat([m, a])
print("\nconcat default      -> index unique:", plain.index.is_unique, "(labels repeat: both files start
at 0)")
fixed = pd.concat([m, a], ignore_index=True)
print("ignore_index=True   -> index unique:", fixed.index.is_unique, "| index range:",
fixed.index.min(), "-", fixed.index.max())
keyed = pd.concat([m, a], keys=["Mar", "Apr"])
print("keys=['Mar','Apr']  -> MultiIndex levels:", keyed.index.names, "| rows per key:",
keyed.groupby(level=0).size().to_dict())

# Column handling: files with different columns
a_extra = a.assign(Comment="checked")
print("\nColumns differ -> outer (default):", pd.concat([m, a_extra],
ignore_index=True).columns.tolist()[-2:],
      "| Comment NaN in", int(pd.concat([m, a_extra],
ignore_index=True)["Comment"].isna().sum()), "rows")
print("                  inner join       :", pd.concat([m, a_extra], join="inner",
ignore_index=True).columns.tolist()[-2:])

# Verification: nothing lost, nothing duplicated
key = ["Machine_ID", "Date", "Shift"]
print("\nRow count equals original  :", len(fixed) == len(prod))
print("No duplicated keys         :", not fixed.duplicated(key).any())
print("Same records as original   :",
fixed.sort_values(key).reset_index(drop=True).equals(prod.sort_values(key).reset_index(drop=Tru
e)))
```

Output

```
March rows: 466 | April rows: 482 | sum: 948 | original: 948

concat default      -> index unique: False (labels repeat: both files start at 0)
ignore_index=True   -> index unique: True | index range: 0 - 947
keys=['Mar','Apr']  -> MultiIndex levels: [None, None] | rows per key: {'Apr': 482, 'Mar': 466}

Columns differ -> outer (default): ['Defect_Rate', 'Comment'] | Comment NaN in 466 rows
                  inner join       : ['Defects_Imputed', 'Defect_Rate']

Row count equals original  : True
No duplicated keys         : True
Same records as original   : True
```

concat stacks the frames vertically. By default the original index labels are kept, so labels repeat (each file starts at 0) – ignore_index=True creates a new unique index, and keys=[...] adds a label saying which file each row came from. Columns are matched by name: with join='outer' (default) the union of columns is kept and missing entries become NaN; with join='inner' only common columns remain. The checks show that 948 rows were split and recombined without loss or duplication.

Step 3 – Group by Line and by Machine_ID (agg with named outputs)

Python code

```
agg_spec = dict(total_output=("Output_Units", "sum"),
                mean_defect_rate=("Defect_Rate", "mean"),
                max_defects=("Defects", "max"),
                records=("Defect_Rate", "count"))
by_line = prod.groupby("Line").agg(**agg_spec)
by_machine = prod.groupby("Machine_ID").agg(**agg_spec)
print("BY LINE"); print(by_line.round(4))
print("\nBY MACHINE"); print(by_machine.round(4))
print("\nRecords add up:", int(by_line.records.sum()), "=", int(by_machine.records.sum()), "=",
len(prod))
```

Output

```
BY LINE
     total_output  mean_defect_rate  max_defects  records
Line
A         170377           0.0385         43     356
B         157768           0.0340         35     331
C         124683           0.0362         48     261

BY MACHINE
          total_output  mean_defect_rate  max_defects  records
Machine_ID
M01              40955            0.0375           30       86
M02              42793            0.0467           43       89
M03              45726            0.0439           33       96
M04              40903            0.0250           23       85
M05              39176            0.0282           23       82
M06              38803            0.0475           35       81
M07              41813            0.0170           19       88
M08              37976            0.0448           35       80
M09              40483            0.0447           35       85
M10              42408            0.0342           27       88
M11              41392            0.0289           26       87
M99                400            0.1200           48        1

Records add up: 948 = 948 = 948
```

Step 4 – Pivot table of mean defect rate by Line and Shift

Python code

```
pivot = prod.pivot_table(values="Defect_Rate", index="Line", columns="Shift",
                         aggfunc="mean", margins=True, margins_name="All")[["Morning", "Evening",
"Night", "All"]]
print(pivot.round(4))
```

```
gb = prod.groupby(["Line", "Shift"])["Defect_Rate"].mean()
print("\nEquivalent groupby result (long format):"); print(gb.round(4).head(6))
print("\nSame numbers:", np.allclose(gb.unstack()[["Morning", "Evening", "Night"]].to_numpy(),
                                    pivot.loc[["A", "B", "C"], ["Morning", "Evening", "Night"]].to_numpy()))
```

Output

```
Shift  Morning  Evening   Night     All
Line
A       0.0332   0.0386  0.0449  0.0385
B       0.0302   0.0339  0.0383  0.0340
C       0.0309   0.0331  0.0446  0.0362
All     0.0316   0.0354  0.0426  0.0363

Equivalent groupby result (long format):
Line  Shift
A     Evening    0.0386
      Morning    0.0332
      Night      0.0449
B     Evening    0.0339
      Morning    0.0302
      Night      0.0383
Name: Defect_Rate, dtype: float64

Same numbers: True
```

The pivot table contains the same numbers as groupby(['Line', 'Shift']).mean(). The difference is the shape: groupby returns a long Series with a two-level index (one row per Line–Shift pair), which is convenient for further computation, whereas pivot_table returns a wide table with Shift as columns, which is easier for a person to compare across shifts, and it can add margins (the ‘All’ row/column) and fill empty combinations with NaN. Note that the ‘All’ margin is the mean over records, not the average of the cell means.

Step 5 – Filter groups (not rows): machines above the overall mean defect rate

Python code

```
overall = prod["Defect_Rate"].mean()
print(f"Overall mean defect rate: {overall:.4f}")

above = prod.groupby("Machine_ID").filter(lambda g: g["Defect_Rate"].mean() > overall)
print("filter(): rows kept", len(above), "| machines:", sorted(above["Machine_ID"].unique()))

prod["machine_mean"] = prod.groupby("Machine_ID")["Defect_Rate"].transform("mean")   # group mean on every row
above_t = prod.loc[prod["machine_mean"] > overall]
print("transform(): rows kept", len(above_t), "| same machines:",
      sorted(above_t["Machine_ID"].unique()) == sorted(above["Machine_ID"].unique()))
print(prod.groupby("Machine_ID")["machine_mean"].first().round(4).sort_values(ascending=False).to_string())
```

Output

```
Overall mean defect rate: 0.0363
filter(): rows kept 518 | machines: ['M01', 'M02', 'M03', 'M06', 'M08', 'M09', 'M99']
transform(): rows kept 518 | same machines: True
Machine_ID
M99    0.1200
M06    0.0475
M02    0.0467
M08    0.0448
M09    0.0447
M03    0.0439
M01    0.0375
M10    0.0342
M11    0.0289
M05    0.0282
M04    0.0250
M07    0.0170
```

filter() keeps or discards entire groups based on a group-level condition (here the machine's mean defect rate), unlike a row filter, which would keep individual days above the mean. transform() returns the group mean aligned to every row so that the same decision can be made with an ordinary boolean mask and the group statistic can be reused as a column; both approaches select the same machines.

Step 6 – Sort machines (descending defect rate, output as tie-breaker); top 5 and bottom 5

Python code

```
ranked = by_machine.sort_values(["mean_defect_rate", "total_output"], ascending=[False, False],
kind="stable")
print("Top 5:");    print(ranked.head(5).round(4))
print("\nBottom 5:"); print(ranked.tail(5).round(4))

# Stable ordering demonstration with deliberate ties
toy = pd.DataFrame({"machine": ["X1", "X2", "X3", "X4"], "rate": [0.03, 0.05, 0.03, 0.05],
"output": [100, 100, 100, 100]})
print("\nTies in both keys, stable sort keeps the original order:")
print(toy.sort_values(["rate", "output"], ascending=[False, False],
kind="stable")["machine"].tolist())
print("Tie in rate only, output breaks the tie:")
toy.loc[[1, 3], "output"] = [90, 120]
print(toy.sort_values(["rate", "output"], ascending=[False, False],
kind="stable")["machine"].tolist())
```

Output

```
Top 5:
            total_output  mean_defect_rate  max_defects  records
Machine_ID
M99                400            0.1200           48        1
M06              38803             0.0475           35       81
M02              42793             0.0467           43       89
M08              37976             0.0448           35       80
```

```
M09             40483          0.0447         35          85

Bottom 5:
            total_output  mean_defect_rate  max_defects  records
Machine_ID
M10             42408          0.0342         27          88
M11             41392          0.0289         26          87
M05             39176          0.0282         23          82
M04             40903          0.0250         23          85
M07             41813          0.0170         19          88

Ties in both keys, stable sort keeps the original order:
['X2', 'X4', 'X1', 'X3']
Tie in rate only, output breaks the tie:
['X4', 'X2', 'X1', 'X3']
```

sort_values with several keys sorts by the first key and uses the next key only to break ties. A stable sort (kind='stable') keeps rows that are equal on all keys in their original order, which makes rankings reproducible. The first toy example shows X2 before X4 and X1 before X3 (original order retained); in the second, output breaks the tie so X4 (120 units) comes before X2 (90 units).

Analysis Inquiry – effect of dropping the unmatched records

Three questions a production manager can answer from these outputs: (1) Which machine has the highest average defect rate and needs maintenance first? (2) Which line produces the most output, and which shift has the highest defect rate? (3) Which supervisor's machines have the highest total output? Question 1 is tested below with unmatched records kept (left merge) and dropped (inner merge).

Python code

```
def worst(df, n=3):
    g = df.groupby("Machine_ID").agg(mean_defect_rate=("Defect_Rate", "mean"),
records=("Defect_Rate", "count"))
    return g.sort_values("mean_defect_rate", ascending=False).head(n)

print("Q1 with unmatched records KEPT (left merge):");   print(worst(left).round(4))
print("\nQ1 with unmatched records DROPPED (inner merge):"); print(worst(inner).round(4))
```

```
sup_left  = left.groupby(left["Supervisor"].fillna("(no supervisor)"))["Output_Units"].sum()
sup_inner = inner.groupby("Supervisor")["Output_Units"].sum()
print("\nQ3 total output by supervisor (left, kept):\n", sup_left.to_string())
print("Q3 total output by supervisor (inner, dropped):\n", sup_inner.to_string())
print("\nM99 record: output", int(left.loc[left.Machine_ID == "M99", "Output_Units"].iloc[0]),
    "defects", int(left.loc[left.Machine_ID == "M99", "Defects"].iloc[0]),
    "-> rate", round(48 / 400, 3), "| machine records:", int((left.Machine_ID == "M99").sum()))
```

Output

```
Q1 with unmatched records KEPT (left merge):
            mean_defect_rate  records
Machine_ID
M99                 0.1200        1
M06                 0.0475       81
M02                 0.0467       89

Q1 with unmatched records DROPPED (inner merge):
            mean_defect_rate  records
Machine_ID
M06                 0.0475       81
M02                 0.0467       89
M08                 0.0448       80

Q3 total output by supervisor (left, kept):
 Supervisor
(no supervisor)       400
Anand              125088
Bhavna             167065
Chitra             160275
Q3 total output by supervisor (inner, dropped):
 Supervisor
Anand     125088
Bhavna    167065
```

```
Chitra    160275


M99 record: output 400 defects 48 -> rate 0.12 | machine records: 1
```

The answer to question 1 changes. When the unmatched record is kept, machine M99 appears at the top of the ranking with a defect rate of 12 % – but this rests on a single record (400 units, 48 defects). When unmatched records are dropped, M99 disappears and machine M06 (4.75 % over 81 records) becomes the worst. Question 3 changes too: with a left merge 400 units of output belong to ‘(no supervisor)’; with an inner merge that output is silently lost and total output is understated. Reliability: the conclusion depends on a join decision, not on the data alone. A ranking based on one record is unreliable, and dropping the record hides a data-quality problem (a machine missing from the master table) instead of resolving it. The reliable approach is to keep the unmatched record visible (indicator column), report it, and require a minimum number of records before ranking a machine.

Q5 – Visualise Data to Support a Decision

Setup

The cleaned production data and the grouped summaries from Q3–Q4 are used. A consistent colour per Line and Shift is defined once so that the plots are comparable.

Python code

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt, matplotlib.dates as mdates


plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False,
"figure.dpi": 100})
line_col  = {"A": "#1F3864", "B": "#C55A11", "C": "#548235"}
shift_col = {"Morning": "#8FAADC", "Evening": "#F4B183", "Night": "#404040"}
prod = prod.drop(columns="machine_mean", errors="ignore")
print("Records plotted:", len(prod), "| date range:", prod.Date.min().date(), "to",
prod.Date.max().date())
```

Output

```
Records plotted: 948 | date range: 2026-03-01 to 2026-04-30
```

Plot 1 – Line plot of daily total output for each Line

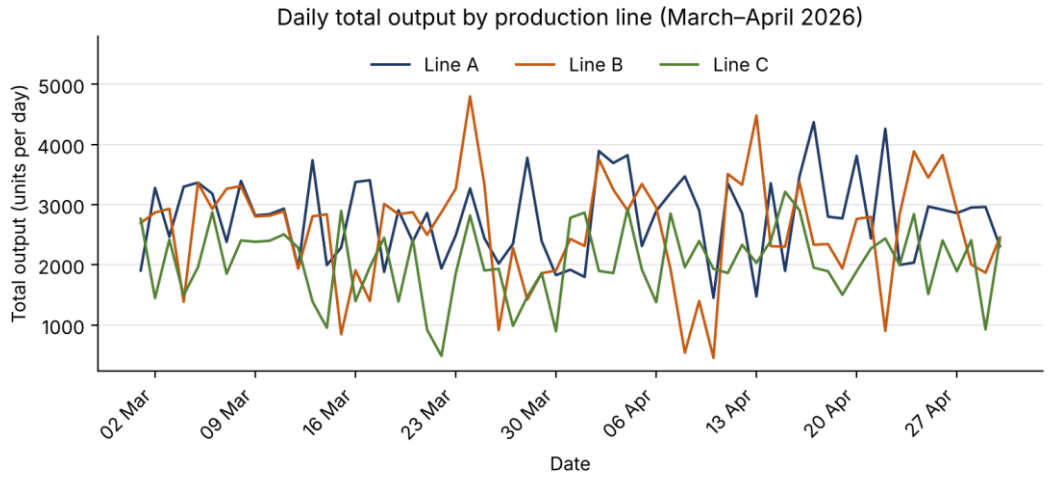
Python code

```
daily = prod.groupby(["Date", "Line"])["Output_Units"].sum().unstack()
fig, ax = plt.subplots(figsize=(8, 3.8))
for ln in daily.columns:
    ax.plot(daily.index, daily[ln], label=f"Line {ln}", color=line_col[ln], lw=1.4)
ax.set_title("Daily total output by production line (March–April 2026)")
ax.set_xlabel("Date"); ax.set_ylabel("Total output (units per day)")
ax.xaxis.set_major_locator(mdates.WeekdayLocator(byweekday=mdates.MO, interval=1))
ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b"))
plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
ax.set_ylim(top=5800); ax.legend(ncol=3, frameon=False, loc="upper center"); ax.grid(axis="y",
alpha=.3)
```

```
fig.tight_layout(); fig.savefig("figs/plot1_line.png", dpi=200); plt.close(fig)
print(daily.describe().loc[["mean", "min", "max"]].round(0))
```

Output

```
Line      A       B       C
mean  2793.0  2586.0  2044.0
min   1452.0   454.0   486.0
max   4371.0  4799.0  3214.0
```



Question answered: how does each line's daily output vary over time and which line produces most? A line plot is appropriate because the x-axis is ordered time and the lines show trend and variability. There is no sustained upward or downward trend, but the daily totals fluctuate strongly because the number of shift records reported per day differs. On average Line A produces most (about 2,793 units per day), then Line B (2,586) and Line C (2,044); Line C is lower because only three of its four machines (M09–M11) have production records. Weekly date ticks keep the dates readable.

Plot 2 – Bar chart of mean defect rate by machine, and histogram of Defect_Rate

Python code

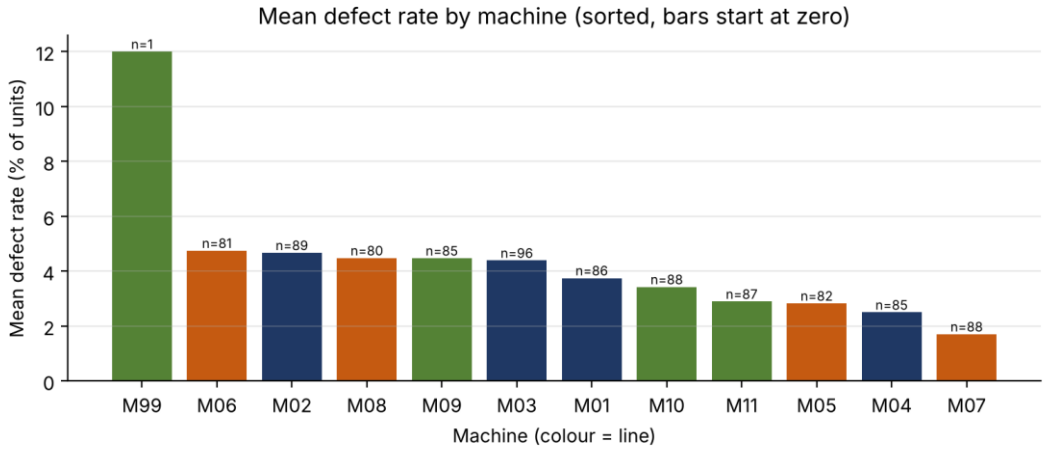
```
ms = prod.groupby("Machine_ID")["Defect_Rate"].agg(["mean", "count"]).sort_values("mean",
ascending=False)
fig, ax = plt.subplots(figsize=(8, 3.6))
ax.bar(ms.index, ms["mean"] * 100, color=[line_col[prod.loc[prod.Machine_ID == m,
"Line"].iloc[0]] for m in ms.index])
for i, (m, row) in enumerate(ms.iterrows()):
    ax.text(i, row["mean"] * 100 + .1, f"n={int(row['count'])}", ha="center", fontsize=7)
ax.set_title("Mean defect rate by machine (sorted, bars start at zero)")
```

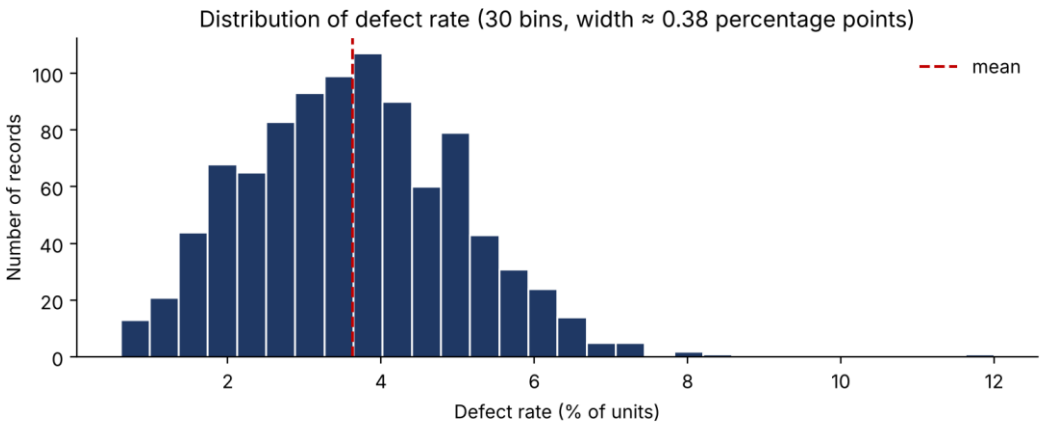
```
ax.set_xlabel("Machine (colour = line)"); ax.set_ylabel("Mean defect rate (% of units)")
ax.grid(axis="y", alpha=.3); fig.tight_layout(); fig.savefig("figs/plot2_bar.png", dpi=200);
plt.close(fig)

n_bins = 30
fig, ax = plt.subplots(figsize=(8, 3.4))
ax.hist(prod["Defect_Rate"] * 100, bins=n_bins, color="#1F3864", edgecolor="white")
ax.axvline(prod["Defect_Rate"].mean() * 100, color="#C00000", ls="--", label="mean")
ax.set_title(f"Distribution of defect rate ({n_bins} bins, width ≈ {(prod.Defect_Rate.max()-
prod.Defect_Rate.min())*100/n_bins:.2f} percentage points)")
ax.set_xlabel("Defect rate (% of units)"); ax.set_ylabel("Number of records");
ax.legend(frameon=False)
fig.tight_layout(); fig.savefig("figs/plot3_hist.png", dpi=200); plt.close(fig)
print(ms.round(4).head(4)); print("Skewness of Defect_Rate:", round(prod.Defect_Rate.skew(), 2))
```

Output

```
            mean  count
Machine_ID
M99       0.1200      1
M06       0.0475     81
M02       0.0467     89
M08       0.0448     80
Skewness of Defect_Rate: 0.44
```





Bar chart – question: which machines have the highest average defect rate? Bars suit comparing one value across discrete categories; they are sorted for easy ranking and start at zero so that bar length is proportional to the value. Counts (n) are printed above the bars so that the single-record machine M99 is not mistaken for a reliable result. Histogram – question: what is the shape of the defect-rate distribution and how much spread is there? Histograms suit one continuous variable. 30 bins were chosen because there are 948 records (about $\sqrt{948} \approx 31$ bins by the square-root rule): this is enough to show the moderately right-skewed shape (skewness 0.44) and the tail without making the bars noisy; the red line marks the mean.

Plot 3 – Box plot by Shift and scatter plot of Output_Units against Defects with a fitted line

Python code

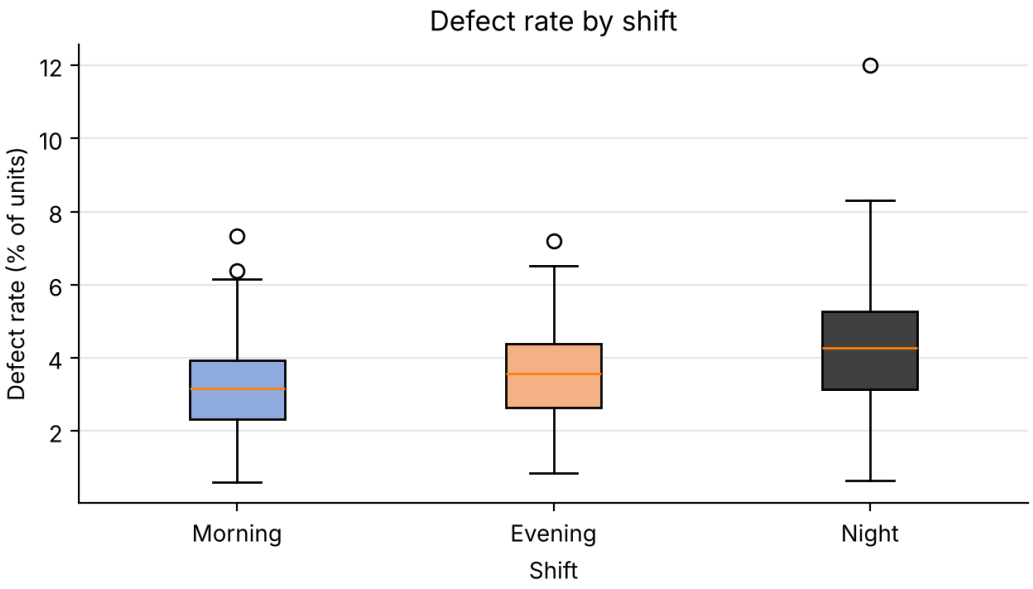
```
fig, ax = plt.subplots(figsize=(6.5, 3.8))
order = ["Morning", "Evening", "Night"]
bp = ax.boxplot([prod.loc[prod.Shift == s, "Defect_Rate"] * 100 for s in order], tick_labels=order,
patch_artist=True)
for patch, s in zip(bp["boxes"], order): patch.set_facecolor(shift_col[s])
ax.set_title("Defect rate by shift"); ax.set_xlabel("Shift"); ax.set_ylabel("Defect rate (% of units)")
ax.grid(axis="y", alpha=.3); fig.tight_layout(); fig.savefig("figs/plot4_box.png", dpi=200);
plt.close(fig)

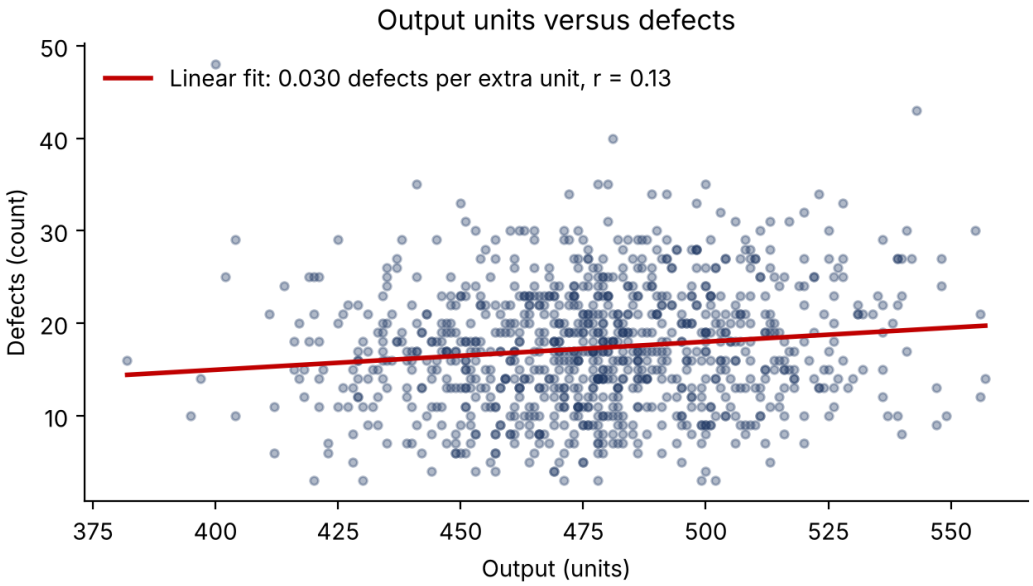
slope, intercept = np.polyfit(prod["Output_Units"], prod["Defects"], 1)
r = np.corrcoef(prod["Output_Units"], prod["Defects"])[0, 1]
xs = np.linspace(prod.Output_Units.min(), prod.Output_Units.max(), 50)
fig, ax = plt.subplots(figsize=(6.5, 3.8))
ax.scatter(prod["Output_Units"], prod["Defects"], s=12, alpha=.35, color="#1F3864")
```

```
ax.plot(xs, slope * xs + intercept, color="#C00000", lw=2, label=f"Linear fit: {slope:.3f} defects per extra unit, r = {r:.2f}")
ax.set_title("Output units versus defects"); ax.set_xlabel("Output (units)"); ax.set_ylabel("Defects (count)")
ax.legend(frameon=False, loc="upper left"); fig.tight_layout(); fig.savefig("figs/plot5_scatter.png", dpi=200); plt.close(fig)
print(prod.groupby("Shift")["Defect_Rate"].agg(["median", "mean", "max"]).loc[order].mul(100).round(2))
print(f"slope = {slope:.4f}, intercept = {intercept:.2f}, r = {r:.3f}")
```

Output

```
        median  mean    max
Shift
Morning    3.16  3.16   7.32
Evening    3.58  3.54   7.20
Night      4.27  4.26  12.00
slope = 0.0304, intercept = 2.81, r = 0.130
```





Box plot – question: do the shifts differ in defect rate, and how spread are they? A box plot compares the median, quartiles and extreme values of one variable across a few groups; the median rises from Morning (3.16 %) to Evening (3.58 %) and Night (4.27 %), and the Night shift also has the widest spread and the largest maximum (12 %, the M99 record). Scatter plot – question: do more units go with more defects? A scatter plot suits the relationship between two numeric variables; transparency (alpha) reduces overplotting of the 948 points. The fitted line has a positive slope (about 0.030 defects per extra unit) but the correlation is weak (r = 0.13): more output means slightly more defects in count, but output explains little of the variation, because the defect rate depends mainly on the machine and the shift.

Plot 4 – Multi-panel figure with a consistent scale

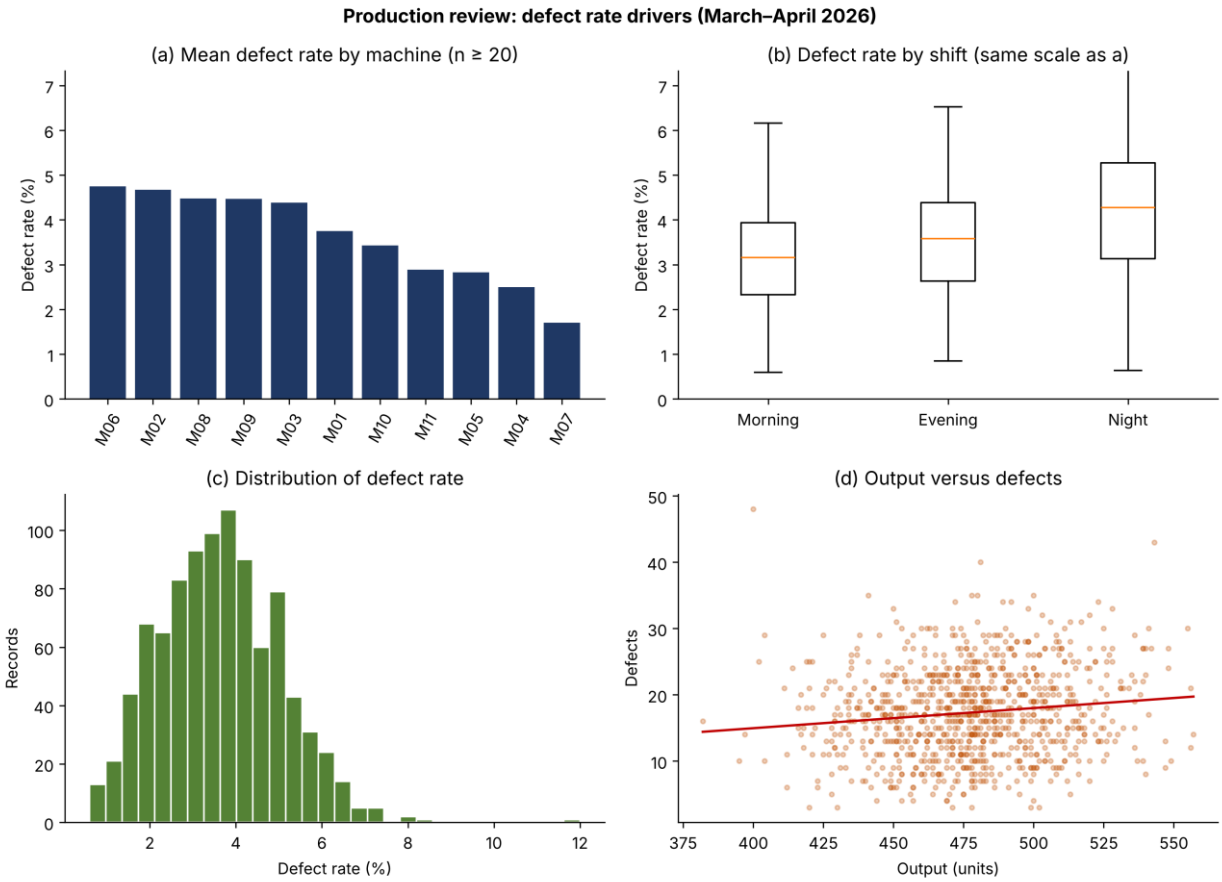
Python code

```
fig, axs = plt.subplots(2, 2, figsize=(11, 8))
# (a) bar chart and (b) box plot share the same y range because defect rates are compared
ms2 = ms[ms["count"] >= 20]          # machines with enough records (excludes the single-record M99)
ymax = prod.Defect_Rate.quantile(.99) * 100 * 1.05
axs[0, 0].bar(ms2.index, ms2["mean"] * 100, color="#1F3864"); axs[0, 0].set_ylim(0, ymax)
axs[0, 0].set_title("(a) Mean defect rate by machine (n ≥ 20)"); axs[0, 0].set_ylabel("Defect rate (%)"); axs[0, 0].tick_params(axis="x", rotation=60)
axs[0, 1].boxplot([prod.loc[prod.Shift == s, "Defect_Rate"] * 100 for s in order], tick_labels=order, showfliers=False)
```

```
axs[0, 1].set_ylim(0, ymax); axs[0, 1].set_title("(b) Defect rate by shift (same scale as a)"); axs[0, 1].set_ylabel("Defect rate (%)")
axs[1, 0].hist(prod["Defect_Rate"] * 100, bins=30, color="#548235", edgecolor="white")
axs[1, 0].set_title("(c) Distribution of defect rate"); axs[1, 0].set_xlabel("Defect rate (%)"); axs[1, 0].set_ylabel("Records")
axs[1, 1].scatter(prod["Output_Units"], prod["Defects"], s=8, alpha=.3, color="#C55A11")
axs[1, 1].plot(xs, slope * xs + intercept, color="#C00000"); axs[1, 1].set_title("(d) Output versus defects")
axs[1, 1].set_xlabel("Output (units)"); axs[1, 1].set_ylabel("Defects")
fig.suptitle("Production review: defect rate drivers (March–April 2026)", fontweight="bold")
fig.tight_layout(); fig.savefig("figs/plot6_panel.png", dpi=300); plt.close(fig)
from PIL import Image
im = Image.open("figs/plot6_panel.png"); print("Saved multi-panel figure:", im.size, "pixels at 300 dpi (11 x 8 in)")
```

Output

```
Saved multi-panel figure: (3300, 2400) pixels at 300 dpi (11 x 8 in)
```



Panels (a) and (b) share the same y-axis starting at zero (M99, with one record, is left out of panel a) because both show defect rate and are meant to be compared; (c) and (d) have different quantities and so are scaled independently. The figure is saved at 300 dpi, which is suitable for printing and for inclusion in a report.

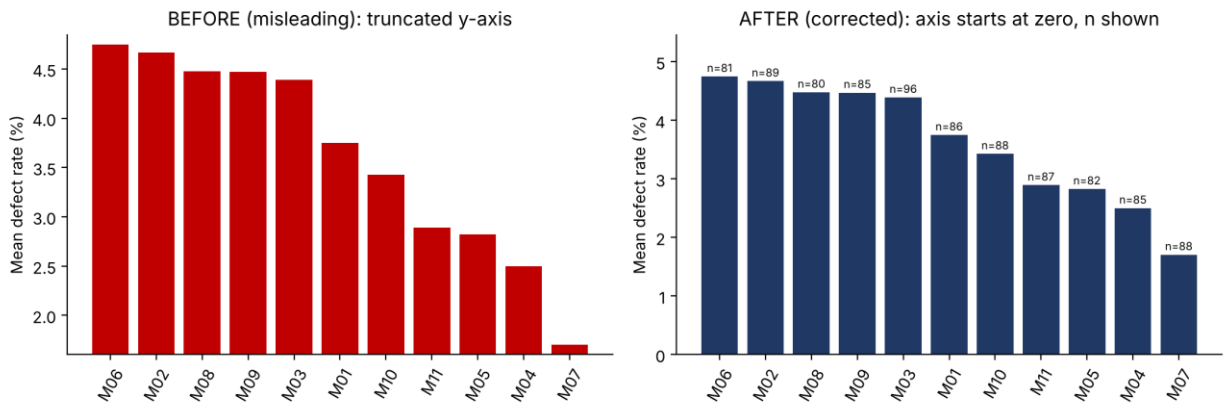
Design Inquiry – a misleading plot and its correction

Python code

```
ms2 = ms[ms["count"] >= 20]
vals = ms2["mean"] * 100
ylo = vals.min() - 0.1                          # truncated axis starts just below the smallest bar
fig, axs = plt.subplots(1, 2, figsize=(11, 3.8))
axs[0].bar(ms2.index, vals, color="#C00000"); axs[0].set_ylim(ylo, vals.max() + .1)
axs[0].set_title("BEFORE (misleading): truncated y-axis"); axs[0].set_ylabel("Mean defect rate (%)")
axs[1].bar(ms2.index, vals, color="#1F3864"); axs[1].set_ylim(0, vals.max() * 1.15)
axs[1].set_title("AFTER (corrected): axis starts at zero, n shown"); axs[1].set_ylabel("Mean defect rate (%)")
for i, (m, row) in enumerate(ms2.iterrows()):
    axs[1].text(i, row["mean"] * 100 + .08, f"n={int(row['count'])}", ha="center", fontsize=7)
for ax in axs: ax.tick_params(axis="x", rotation=60)
fig.tight_layout(); fig.savefig("figs/plot7_before_after.png", dpi=200); plt.close(fig)
print(f"Truncated axis: tallest bar looks {(vals.max()-ylo)/(vals.min()-ylo):.1f}x the shortest bar")
print(f"Actual ratio of the same two means: {vals.max()/vals.min():.2f}x")
```

Output

```
Truncated axis: tallest bar looks 31.4x the shortest bar
Actual ratio of the same two means: 2.79x
```



Which figures could mislead? (1) The bar chart with a truncated axis: if the y-axis starts near the lowest value, small differences look large because bar length no longer represents the value. This is shown on the left – the tallest bar appears many times the shortest – while the corrected version on the right starts at zero (the true ratio is under three) and prints the number of records. (2) A histogram with too few or too many bins can hide the skew or create false peaks, which is why the bin count is stated. (3) The scatter plot can hide overlap (many points with equal integer values), which is reduced by transparency. A plot type that would be misleading for the same data: a pie chart of defect rate by machine, because defect rates are not parts of a whole (they do not sum to 100 %); a 3-D chart, or a line connecting machine IDs, would also suggest an order or continuity between machines that does not exist.

Q6 – Engineer a Reproducible Data Analysis Pipeline

Step 1 – Pipeline organised into functions (DataFrame in, DataFrame out)

Every stage is a function whose only inputs are its arguments; there are no global variables. load_data receives a DataFrame listing the files to read; clean_data, combine_data, summarise_* and plot_summary receive and return DataFrames. A custom PipelineError stops the run with a clear message.

Python code

```
import hashlib, os, tracemalloc, matplotlib.pyplot as plt

REQUIRED = ["Machine_ID", "Date", "Line", "Shift", "Output_Units", "Defects"]
KEY = ["Machine_ID", "Date", "Shift"]

class PipelineError(Exception):
    """Raised when a validation check fails; the pipeline stops with a clear message."""

def check_columns(df, required, stage):
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise PipelineError(f"[{stage}] required column(s) missing: {missing}")

def check_unique_key(df, key, stage):
    n = int(df.duplicated(key).sum())
    if n:
        raise PipelineError(f"[{stage}] {n} duplicated key(s) on {key} after cleaning")

def load_data(sources):                    # sources: DataFrame with a 'path' column
    frames = [pd.read_csv(p, parse_dates=["Date"], dtype={"Output_Units": "Int64", "Defects":
"Int64"})
              for p in sources["path"]]
    out = pd.concat(frames, ignore_index=True)
    check_columns(out, REQUIRED, "load")
    return out
```

```
def clean_data(df, defects_policy="impute"):
    check_columns(df, REQUIRED, "clean")
    d = df.drop_duplicates().copy()
    d["Line"] = d["Line"].str.strip().str.upper()
    check_unique_key(d, KEY, "clean")
    d = d.dropna(subset=["Output_Units"])
    rate = (d["Defects"] / d["Output_Units"]).astype("float64")
    d["Defects_Imputed"] = d["Defects"].isna()
    if defects_policy == "impute":
        med = rate.groupby(d["Machine_ID"]).transform("median")
        d["Defects"] = d["Defects"].fillna((med * d["Output_Units"]).round()).astype("int64")
    else:
        d = d.dropna(subset=["Defects"])
    d = d.astype({"Output_Units": "int64", "Defects": "int64"}).reset_index(drop=True)
    d["Defect_Rate"] = d["Defects"] / d["Output_Units"]
    return d

def combine_data(prod, info):
    check_columns(info, ["Machine_ID", "Supervisor"], "combine")
    n_before = len(prod)
    out = prod.merge(info[["Machine_ID", "Supervisor", "Installed_Year"]], on="Machine_ID",
how="left")
    if len(out) != n_before:
        raise PipelineError(f"[combine] row count changed from {n_before} to {len(out)} after
merge")
    return out

def summarise_machine(df):
    return (df.groupby("Machine_ID").agg(total_output=("Output_Units", "sum"),
            mean_defect_rate=("Defect_Rate", "mean"), max_defects=("Defects", "max"),
            records=("Defect_Rate", "count")).reset_index())

def summarise_line(df):
    return df.groupby(["Line", "Shift"], as_index=False).agg(total_output=("Output_Units", "sum"),
            mean_defect_rate=("Defect_Rate", "mean"))
```

```
def daily_output(df):
    return df.groupby(["Date", "Line"], as_index=False)["Output_Units"].sum()

def plot_summary(summary, out_path):
    s = summary.sort_values("mean_defect_rate", ascending=False)
    fig, ax = plt.subplots(figsize=(7, 3)); ax.bar(s["Machine_ID"], s["mean_defect_rate"] * 100)
    ax.set_xlabel("Machine"); ax.set_ylabel("Mean defect rate (%)"); fig.tight_layout()
    fig.savefig(out_path, dpi=100); plt.close(fig)
    return summary

def run_pipeline(sources, info, out_dir):
    os.makedirs(out_dir, exist_ok=True)
    clean = clean_data(load_data(sources))
    merged = combine_data(clean, info)
    outputs = {"machine_summary": summarise_machine(merged), "line_shift_summary":
summarise_line(merged),
               "daily_output": daily_output(merged)}
    for name, df in outputs.items():
        df.to_csv(os.path.join(out_dir, name + ".csv"), index=False)
    plot_summary(outputs["machine_summary"], os.path.join(out_dir, "machine_defect_rate.png"))
    return outputs

info = pd.read_csv("machine_info.csv")
sources = pd.DataFrame({"path": ["production_raw.csv"]})
out1 = run_pipeline(sources, info, "run1")
for k, v in out1.items():
    print(f"{k:20s} shape={v.shape}")
print(out1["machine_summary"].head(3).round(4).to_string(index=False))
```

Output

```
machine_summary      shape=(12, 5)
line_shift_summary   shape=(9, 4)
daily_output         shape=(183, 3)
Machine_ID  total_output  mean_defect_rate  max_defects  records
       M01         40955            0.0375           30       86
```

```
M02     42793     0.0467     43     89
M03     45726     0.0439     33     96
```

The pipeline has five stages – load → clean → combine → summarise → plot – and each stage returns a DataFrame that the next stage consumes. Because each function only uses its arguments, it can be tested alone and gives the same result whenever it is called with the same input.

Step 2 – Validation checks with example failures

Python code

```
raw = pd.read_csv("production_raw.csv", parse_dates=["Date"], dtype={"Output_Units": "Int64", "Defects": "Int64"})

def attempt(label, fn):
    try:
        fn(); print(f"{label}: no error")
    except PipelineError as e:
        print(f"{label}: STOPPED -> {e}")

# 1. Required column missing
attempt("Check 1 (missing column 'Shift')", lambda: clean_data(raw.drop(columns="Shift")))

# 2. Key duplicated after cleaning (same machine/date/shift, different values -> not an exact duplicate)
bad = pd.concat([raw, raw.iloc[[0]].assign(Output_Units=999)], ignore_index=True)
attempt("Check 2 (key duplicated, differing values)", lambda: clean_data(bad))

# 3. Row count changes after merge (Machine_ID duplicated in Machine_Info)
clean_ok = clean_data(raw)
info_bad = pd.concat([info, info.iloc[[0]]], ignore_index=True)
attempt("Check 3 (duplicate key in Machine_Info)", lambda: combine_data(clean_ok, info_bad))

print("\nWhich checks caught REAL problems in the real data?")
print("  Raw data: duplicated keys before cleaning =", int(raw.duplicated(KEY).sum()), "(exact duplicates, removed by clean_data)")
print("  Raw data: Line spellings before cleaning  =", raw['Line'].nunique(), "-> after:", clean_ok['Line'].nunique())
```

```
print("  Required columns missing in real file    :", [c for c in REQUIRED if c not in raw.columns])
print("  Row count change in the real merge       :", len(combine_data(clean_ok, info)) -
len(clean_ok))
print("  Unmatched production records (no check!) :", int(combine_data(clean_ok,
info)["Supervisor"].isna().sum()))
```

Output

```
Check 1 (missing column 'Shift'): STOPPED -> [clean] required column(s) missing: ['Shift']
Check 2 (key duplicated, differing values): STOPPED -> [clean] 1 duplicated key(s) on
['Machine_ID', 'Date', 'Shift'] after cleaning
Check 3 (duplicate key in Machine_Info): STOPPED -> [combine] row count changed from 948 to
1034 after merge


Which checks caught REAL problems in the real data?
  Raw data: duplicated keys before cleaning = 15 (exact duplicates, removed by clean_data)
  Raw data: Line spellings before cleaning  = 10 -> after: 3
  Required columns missing in real file    : []
  Row count change in the real merge       : 0
  Unmatched production records (no check!) : 1
```

Selection Inquiry. Checks that caught real problems: the key-uniqueness check is meaningful because the raw file really contained 15 duplicated records (they are exact copies, so clean_data removes them first and the check then passes on the cleaned data; without cleaning it would stop the run). The column check and the merge row-count check did not fire on the real data, but they are cheap insurance and they do fire on the simulated failures above – a renamed column in next week's file or a duplicated machine in Machine_Info would otherwise produce silent errors. They are 'unnecessary' only for this week's data, not for the weekly process. One real problem was NOT caught by any check: one production record (M99) has no match in Machine_Info and passes the left merge with Supervisor = NaN.

If the checks were removed: output totals and defect rates would be overstated by duplicate records, a renamed or missing column could silently produce empty or wrong summaries, and a duplicated Machine_Info row would multiply that machine's production records, inflating its total output and distorting the ranking of machines and the supervisor totals. The conclusions about machine and line rankings and total output would then be unreliable.

Step 3 – Three ways of computing a derived column on 100,000 rows

Python code

```
big = clean_ok.sample(n=100_000, replace=True, random_state=0).reset_index(drop=True)
print("Rows:", len(big))

def by_loop(df):
    out = []
    for i in range(len(df)):
        out.append(df["Defects"].iat[i] / df["Output_Units"].iat[i])
    return pd.Series(out, index=df.index)

def by_apply(df):
    return df.apply(lambda row: row["Defects"] / row["Output_Units"], axis=1)

def by_vector(df):
    return df["Defects"] / df["Output_Units"]

results = {}
for name, fn in [("Python loop", by_loop), ("DataFrame.apply", by_apply), ("Vectorised",
by_vector)]:
    t0 = time.perf_counter(); r = fn(big); t = time.perf_counter() - t0
    tracemalloc.start(); fn(big); peak = tracemalloc.get_traced_memory()[1]; tracemalloc.stop()
    results[name] = (t, peak / 1e6, r)

print(f"{'Method':18s}{'Time (s)':>10s}{'Peak extra memory (MB)':>26s}{'Speed vs
vectorised':>22s}")
tv = results["Vectorised"][0]
for k, (t, m, _) in results.items():
    print(f"{k:18s}{t:10.4f}{m:26.2f}{t/tv:21.0f}x")
v = results["Vectorised"][2]
print("\nAll three give the same values:",
      all(np.allclose(r.to_numpy(dtype=float), v.to_numpy(dtype=float)) for _, _, r in
results.values()))
```

Output

```
Rows: 100000
Method            Time (s)    Peak extra memory (MB)   Speed vs vectorised
Python loop         2.0895                  9.00              2475x
DataFrame.apply     0.4906                 41.04               581x
Vectorised          0.0008                  0.93                1x

All three give the same values: True
```

All three methods give identical values, but the vectorised expression is by far the fastest and uses the least temporary memory. The loop and apply(axis=1) run Python code once per row and create a temporary object (a Series) for every row; the vectorised division runs in compiled code over whole columns.

Step 4 – Reduce DataFrame memory by choosing suitable dtypes

Python code

```
def optimise_dtypes(df):
    d = df.copy()
    for c in ["Machine_ID", "Line", "Shift"]:
        d[c] = d[c].astype("category")
    d["Output_Units"] = d["Output_Units"].astype("int16")
    d["Defects"] = d["Defects"].astype("int16")
    d["Defect_Rate"] = d["Defect_Rate"].astype("float32")
    return d


assert big["Output_Units"].max() < 32767 and big["Defects"].max() < 32767
small = optimise_dtypes(big)
before_mb = big.memory_usage(deep=True).sum() / 1e6
after_mb = small.memory_usage(deep=True).sum() / 1e6
print(pd.DataFrame({"before (MB)": big.memory_usage(deep=True) / 1e6, "after (MB)":
small.memory_usage(deep=True) / 1e6}).round(3))
print(f"\nTotal before: {before_mb:.2f} MB | after: {after_mb:.2f} MB | reduction: {100*(1-
after_mb/before_mb):.0f} %")
print("Values preserved (integers exact):", bool((big["Output_Units"] ==
```

```
small["Output_Units"]).all() and (big["Defects"] == small["Defects"]).all()))
print("Max |difference| in Defect_Rate (float32):", float((big["Defect_Rate"] -
small["Defect_Rate"]).abs().max()))
```

Output

```
                before (MB)  after (MB)
Index                 0.000       0.000
Machine_ID            5.200       0.101
Date                  0.800       0.800
Line                  5.000       0.100
Shift                 5.538       0.100
Output_Units          0.800       0.200
Defects               0.800       0.200
Defects_Imputed       0.100       0.100
Defect_Rate           0.800       0.400

Total before: 19.04 MB | after: 2.00 MB | reduction: 89 %
Values preserved (integers exact): True
Max |difference| in Defect_Rate (float32): 3.5187254499025045e-09
```

Text columns with few distinct values (Machine_ID, Line, Shift) are stored once each as a category and referenced by small integer codes; counts that never exceed a few hundred fit into int16 instead of int64; float32 halves the size of the rate column at the cost of about 7 significant digits. The values are unchanged (the integers exactly, the rate to within about 10^{-9}).

Step 5 – Reproducibility: run twice, then once on a changed input

Python code

```
def digest(outputs):
   return {k: hashlib.md5(pd.util.hash_pandas_object(v,
index=True).to_numpy().tobytes()).hexdigest()[:10]
           for k, v in outputs.items()}

out2 = run_pipeline(sources, info, "run2")
d1, d2 = digest(out1), digest(out2)
```

```
print("Run 1 vs Run 2 in-memory digests identical:", d1 == d2)
for name in out1:
    f1 = hashlib.md5(open(f"run1/{name}.csv", "rb").read()).hexdigest()
    f2 = hashlib.md5(open(f"run2/{name}.csv", "rb").read()).hexdigest()
    print(f"  {name:20s} file md5 run1={f1[:8]} run2={f2[:8]}  identical={f1 == f2}")

# Changed input: M03's April output is 25 % higher in the new file
changed = raw.copy()
sel = (changed["Machine_ID"] == "M03") & (changed["Date"] >= "2026-04-01") &
changed["Output_Units"].notna()
changed.loc[sel, "Output_Units"] = (changed.loc[sel, "Output_Units"] *
1.25).round().astype("Int64")
changed.to_csv("production_changed.csv", index=False)
out3 = run_pipeline(pd.DataFrame({"path": ["production_changed.csv"]}), info, "run3")

print("\nChanged input (", int(sel.sum()), "rows of M03 altered ) -> which outputs changed?")
for name in out1:
    same = out1[name].equals(out3[name])
    print(f"  {name:20s} changed = {not same}")
m1, m3 = out1["machine_summary"].set_index("Machine_ID"),
out3["machine_summary"].set_index("Machine_ID")
diff_rows = [m for m in m1.index if not m1.loc[m].equals(m3.loc[m])]
print("Machines whose summary changed:", diff_rows)
cmp_ = m1.loc[diff_rows, ["total_output", "mean_defect_rate"]].join(m3.loc[diff_rows,
["total_output", "mean_defect_rate"]], lsuffix="_before", rsuffix="_after")
print(cmp_.round(4).to_string())
l1, l3 = out1["line_shift_summary"], out3["line_shift_summary"]
print("Line/shift rows changed:", int((l1.round(10) != l3.round(10)).any(axis=1).sum()), "of",
len(l1), "(only Line A, M03's line)")
```

Output

```
Run 1 vs Run 2 in-memory digests identical: True
  machine_summary      file md5 run1=725020d2 run2=725020d2  identical=True
  line_shift_summary   file md5 run1=0000c13d run2=0000c13d  identical=True
```

```
  daily_output         file md5 run1=67669718 run2=67669718  identical=True

Changed input ( 49 rows of M03 altered ) -> which outputs changed?
  machine_summary      changed = True
  line_shift_summary   changed = True
  daily_output         changed = True
Machines whose summary changed: ['M03']
          total_output_before  mean_defect_rate_before  total_output_after  mean_defect_rate_after
Machine_ID
M03                   45726                  0.0439               51342                  0.0395
Line/shift rows changed: 3 of 9 (only Line A, M03's line)
```

Running the pipeline twice on the same file gives identical results – equal in-memory digests and identical output files – because there is no randomness and no dependence on hidden state. On the changed input only the outputs that depend on the changed records change: the machine summary for M03 only, the Line A rows of the line/shift summary and the April rows of the daily output for Line A; every other machine's row is identical. This shows the effect of each change is visible and traceable.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate a Data Analysis Solution

A. Diagnose the Data or Computational Bottleneck

Selected limitation: the Q4 result that changes when unmatched records are dropped, which the Q6 pipeline accepts silently. The evidence is regenerated from the original analysis for four scenarios: unmatched records kept (left merge) or dropped (inner merge), crossed with missing Defects filled or dropped.

Python code

```
def original_ranking(df, how):
    """Original Q4 logic: merge, then rank machines by mean defect rate with no safeguards."""
    m = df.merge(info[["Machine_ID", "Supervisor"]], on="Machine_ID", how=how)
    g = m.groupby("Machine_ID").agg(mean_defect_rate=("Defect_Rate", "mean"),
records=("Defect_Rate", "count"))
    return g.sort_values("mean_defect_rate", ascending=False)

scenarios = [(h, p) for h in ["left", "inner"] for p in ["impute", "drop"]]
base = {}
print(f"{'Scenario (merge, Defects)':28s}{'rows':>6s}{'top-1':>7s}  top-5")
for how, pol in scenarios:
    r = original_ranking(clean_data(raw, pol), how)
    base[(how, pol)] = r
    print(f"{how + ', ' + pol:28s}{int(r.records.sum()):6d}{r.index[0]:>7s}  {list(r.index[:5])}")

ref = set(base[("left", "impute")].index[:5])
agree = sum(set(r.index[:5]) == ref for r in base.values())
print(f"\nScenarios whose top-5 equals the reference (left, impute): {agree} of 4")
m99 = base[("left", "impute")].loc["M99"]
print(f"Cause: M99 has {int(m99.records)} record, mean defect rate {m99.mean_defect_rate:.3f} "
      f"(next highest: {base[('left','impute')].iloc[1].mean_defect_rate:.3f} over
{int(base[('left','impute')].iloc[1].records)} records)")
```

```
print("Machine records ranked by number of records (smallest 3):")
print(base[("left", "impute")].sort_values("records").head(3).round(4).to_string())
```

Output

```
Scenario (merge, Defects)     rows  top-1  top-5
left, impute                   948    M99  ['M99', 'M06', 'M02', 'M08', 'M09']
left, drop                     923    M99  ['M99', 'M06', 'M02', 'M08', 'M09']
inner, impute                  947    M06  ['M06', 'M02', 'M08', 'M09', 'M03']
inner, drop                    922    M06  ['M06', 'M02', 'M08', 'M09', 'M03']

Scenarios whose top-5 equals the reference (left, impute): 2 of 4
Cause: M99 has 1 record, mean defect rate 0.120 (next highest: 0.047 over 81 records)
Machine records ranked by number of records (smallest 3):
            mean_defect_rate  records
Machine_ID
M99                   0.1200        1
M08                   0.0448       80
M06                   0.0475       81
```

Diagnosis. The ranking is not stable across reasonable analyst choices: whether unmatched records are kept or dropped decides which machine is reported as the worst. The cause is a combination of a merge decision (an unmatched record) and a small-sample statistic: one record from machine M99 with a 12 % defect rate becomes the ‘worst machine’ because a mean based on one record is given the same standing as a mean based on about 85 records. The Q6 pipeline made it worse because it accepted the unmatched record silently (Supervisor = NaN) and gave no report of it. This is therefore primarily a merge-and-grouping problem, not a computational one.

B. Define the Design Objective

State a measurable redesign objective derived from the diagnosis. Examples include reducing runtime by a stated factor, cutting memory by a stated percentage, making a conclusion stable across cleaning options, or detecting a stated class of bad input automatically. Specify the baseline, target and acceptable trade-offs before redesign.

Python code

```
objective = pd.DataFrame({
   "Item": ["Metric", "Baseline (original)", "Target (redesign)", "Detection target", "Acceptable trade-offs"],
   "Definition": [
      "Stability = number of the 4 scenarios whose top-5 machine list equals the reference list",
      f"{agree} of 4 scenarios stable; top-1 machine differs between merge choices",
      "4 of 4 scenarios stable; identical top-1 and top-5 in every scenario",
      "100 % of unmatched production records and unmatched Machine_Info rows reported, none lost silently",
      "Run time at most 3x the original (milliseconds); machines with < 20 records are reported separately, not ranked"]})
import textwrap
for _, row in objective.iterrows():
   print(textwrap.fill(row["Definition"], width=96, initial_indent=f"{row['Item']:22s}: ", subsequent_indent=" " * 24))
```

Output

```
Metric                 : Stability = number of the 4 scenarios whose top-5 machine list equals
                         the reference list
Baseline (original)    : 2 of 4 scenarios stable; top-1 machine differs between merge choices
Target (redesign)      : 4 of 4 scenarios stable; identical top-1 and top-5 in every scenario
Detection target       : 100 % of unmatched production records and unmatched Machine_Info rows
                         reported, none lost silently
Acceptable trade-offs : Run time at most 3x the original (milliseconds); machines with < 20
                         records are reported separately, not ranked
```

The objective is measurable (stability count, reporting completeness, run-time limit) and directly tied to the diagnosis. The minimum record count of 20 is a design parameter: it is chosen as about a quarter of a typical machine's record count (≈ 80), so that a machine is ranked only when its mean rests on a reasonable number of days.

C. Design the Revised Analysis Strategy

Changes and why. (1) Explicit merge validation: merge with validate='m:1' and indicator=True, so duplicated Machine_Info keys raise an error and every record is labelled matched or unmatched – this addresses silent acceptance. (2) Quarantine report: unmatched production records and unmatched Machine_Info rows are written to a separate table and counted (matched + unmatched = total rows is asserted) – this makes the merge decision visible instead of hidden. A threshold stops the run if the unmatched share is above 1 %. (3) Minimum evidence rule: a machine is ranked only if it has at least min_records = 20 records; this is applied with groupby/transform in one vectorised pass and addresses the small-sample instability. Machines below the limit are listed as 'insufficient data' with their record count. (4) Row accounting after every step.

Additional computation / memory: one extra boolean column (_merge) and one group-size column; effectively negligible. New limitations: a genuinely bad machine with fewer than 20 records is not ranked (it is flagged for follow-up instead); the threshold of 20 is a judgement; an alternative, empirical-Bayes shrinkage of the mean towards the overall mean, would avoid a hard cut-off but is harder to explain.

Python code

```
def robust_ranking(df, info, min_records=20, max_unmatched_share=0.01):
    # 1. explicit merge validation with indicator
    m = df.merge(info[["Machine_ID", "Supervisor"]], on="Machine_ID", how="outer",
                 validate="m:1", indicator=True)
    unmatched_prod = m[m["_merge"] == "left_only"]
    unmatched_info = m[m["_merge"] == "right_only"]
    matched = m[m["_merge"] == "both"]
    # 2. row accounting and threshold check
    assert len(matched) + len(unmatched_prod) == len(df), "records lost or duplicated"
    share = len(unmatched_prod) / len(df)
    if share > max_unmatched_share:
        raise PipelineError(f"unmatched share {share:.1%} exceeds limit
{max_unmatched_share:.0%}")
    # 3. rank only machines with enough evidence (production records, matched or not)
    prod_rows = m[m["_merge"] != "right_only"]
    g = prod_rows.groupby("Machine_ID").agg(mean_defect_rate=("Defect_Rate", "mean"),
```

```
records=("Defect_Rate", "count"))
    ranked = g[g["records"] >= min_records].sort_values("mean_defect_rate", ascending=False)
    insufficient = g[g["records"] < min_records]
    report = dict(unmatched_records=len(unmatched_prod),
unmatched_machines=unmatched_info["Machine_ID"].tolist(),
                  insufficient=insufficient)
    return ranked, report

ranked, report = robust_ranking(clean_data(raw), info)
print("Quarantine / reporting output:")
print("  unmatched production records :", report["unmatched_records"])
print("  Machine_Info rows with no production:", report["unmatched_machines"])
print("  machines with insufficient data (<20 records):");
print(report["insufficient"].round(4).to_string())
print("\nTop 5 ranked machines:"); print(ranked.head(5).round(4).to_string())
```

Pseudocode

```
FUNCTION robust_ranking(production, machine_info, min_records, max_unmatched_share):
    merged   <- OUTER MERGE production WITH machine_info ON Machine_ID
                (validate = many-to-one, indicator = TRUE)     # fails on duplicate master keys
    unmatched_prod <- rows WHERE indicator = left_only
    unmatched_info <- rows WHERE indicator = right_only
    ASSERT count(matched) + count(unmatched_prod) = count(production)   # nothing lost
    IF count(unmatched_prod) / count(production) > max_unmatched_share:
        STOP with message                                  # too many unmatched records
    stats  <- GROUP production rows BY Machine_ID: mean(Defect_Rate), count(records)
    ranked <- stats WHERE records >= min_records, SORT BY mean(Defect_Rate) DESC
    insufficient <- stats WHERE records < min_records
    RETURN ranked, report(unmatched_prod, unmatched_info, insufficient)
```

Output

```
Quarantine / reporting output:
  unmatched production records : 1
  Machine_Info rows with no production: ['M12']
  machines with insufficient data (<20 records):
```

```
         mean_defect_rate  records
Machine_ID
M99                  0.12        1


Top 5 ranked machines:
         mean_defect_rate  records
Machine_ID
M06                0.0475       81
M02                0.0467       89
M08                0.0448       80
M09                0.0447       85
M03                0.0439       96
```

A workflow diagram of the revised procedure is shown below.

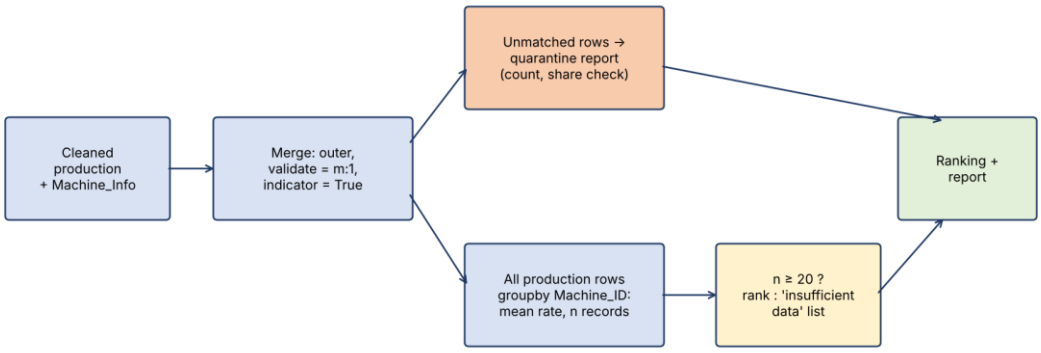
Workflow diagram of the revised strategy

Python code

```
from matplotlib.patches import FancyBboxPatch
fig, ax = plt.subplots(figsize=(9, 3.6)); ax.axis("off"); ax.set_xlim(0, 12); ax.set_ylim(0, 5)
def box(x, y, w, h, text, fc="#D9E2F3"):
    ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.05", fc=fc, ec="#1F3864"))
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", fontsize=8)
def arrow(x1, y1, x2, y2):
    ax.annotate("", xy=(x2, y2), xytext=(x1, y1), arrowprops=dict(arrowstyle="->",
color="#1F3864"))
box(0.1, 2.0, 1.8, 1.2, "Cleaned\nproduction\n+ Machine_Info")
box(2.5, 2.0, 2.2, 1.2, "Merge: outer,\nvalidate = m:1,\nindicator = True")
box(5.4, 3.4, 2.2, 1.2, "Unmatched rows →\nquarantine report\n(count, share check)",
"#F8CBAD")
box(5.4, 0.4, 2.2, 1.2, "All production rows\ngroupby Machine_ID:\nmean rate, n records")
box(8.3, 0.4, 1.8, 1.2, "n ≥ 20 ?\nrank : 'insufficient\ndata' list", "#FFF2CC")
box(10.4, 2.0, 1.5, 1.2, "Ranking +\nreport", "#E2F0D9")
arrow(1.9, 2.6, 2.5, 2.6); arrow(4.7, 2.9, 5.4, 3.9); arrow(4.7, 2.3, 5.4, 1.1)
arrow(7.6, 1.0, 8.3, 1.0); arrow(10.1, 1.0, 10.9, 2.0); arrow(7.6, 3.9, 10.9, 3.2)
fig.tight_layout(); fig.savefig("figs/q7_workflow.png", dpi=200); plt.close(fig)
print("Workflow diagram saved")
```

Output

```
Workflow diagram saved
```



D. Predict Before Implementing

Prediction recorded before running the validation:

(1) Stability will rise from 2 of 4 to 4 of 4 scenarios, because M99 (1 record) is excluded by the minimum-evidence rule whether or not the merge keeps it, and the remaining machines are the same in all scenarios. The top-1 machine will be M06 in every scenario. (2) The ranked values for machines with ≥ 20 records will be identical to the original ranking with M99 removed (same arithmetic, only the selection differs). (3) The unmatched report will contain exactly 1 production record (M99) and 1 Machine_Info machine (M12). (4) Run time will be roughly 1–3 times the original (a few extra milliseconds). (5) The one measure expected to improve is ranking stability across the four scenarios; no other numbers should change.

E. Implement and Validate the Redesign

Python code

```
print(f"{'Scenario (merge, Defects)':28s}{'top-1':>7s}  top-5 (redesign)")
new = {}
for how, pol in scenarios:
    d = clean_data(raw, pol)
    if how == "inner":                          # analyst chooses to drop unmatched records first
        d = d[d["Machine_ID"].isin(info["Machine_ID"])]
```

```
    r, rep = robust_ranking(d, info)
    new[(how, pol)] = r
    print(f"{how + ', ' + pol:28s}{r.index[0]:>7s}  {list(r.index[:5])}")

ref_new = set(new[("left", "impute")].index[:5])
agree_new = sum(set(r.index[:5]) == ref_new for r in new.values())
print(f"\nStability: original {agree} of 4  ->  redesign {agree_new} of 4")

# Same results where they should be the same
orig_no99 = base[("left", "impute")].drop("M99")
common = orig_no99.index
print("Machine means identical to original (excluding M99):",
      bool(np.allclose(orig_no99.loc[common, "mean_defect_rate"], new[("left",
"impute")].loc[common, "mean_defect_rate"])))
print("Row accounting: matched + unmatched =", len(clean_data(raw)), "(asserted inside
robust_ranking)")
r, rep = robust_ranking(clean_data(raw), info)
print("Unmatched report -> records:", rep["unmatched_records"], "| machines without production:",
rep["unmatched_machines"])

# Run time
d0 = clean_data(raw)
t_o = min(__import__("timeit").repeat(lambda: original_ranking(d0, "left"), number=20, repeat=5))
/ 20
t_n = min(__import__("timeit").repeat(lambda: robust_ranking(d0, info), number=20, repeat=5)) /
20
print(f"Run time: original {t_o*1000:.2f} ms | redesign {t_n*1000:.2f} ms | ratio {t_n/t_o:.2f}x")

# Failure test: the design stops on bad input
try:
    robust_ranking(d0, pd.concat([info, info.iloc[[0]]], ignore_index=True))
except Exception as e:
    print("Duplicate Machine_Info key ->", type(e).__name__, ":", str(e).split(chr(10))[0])
try:
    robust_ranking(d0, info.iloc[:6])
```

```
except PipelineError as e:
    print("Too many unmatched records ->", e)
```

Output

```
Scenario (merge, Defects)     top-1  top-5 (redesign)
left, impute                  M06  ['M06', 'M02', 'M08', 'M09', 'M03']
left, drop                    M06  ['M06', 'M02', 'M08', 'M09', 'M03']
inner, impute                 M06  ['M06', 'M02', 'M08', 'M09', 'M03']
inner, drop                   M06  ['M06', 'M02', 'M08', 'M09', 'M03']

Stability: original 2 of 4  ->  redesign 4 of 4
Machine means identical to original (excluding M99): True
Row accounting: matched + unmatched = 948 (asserted inside robust_ranking)
Unmatched report -> records: 1 | machines without production: ['M12']
Run time: original 3.94 ms | redesign 9.27 ms | ratio 2.35x
Duplicate Machine_Info key -> MergeError : Merge keys are not unique in right dataset; not a many-to-one merge
Too many unmatched records -> unmatched share 45.3% exceeds limit 1%
```

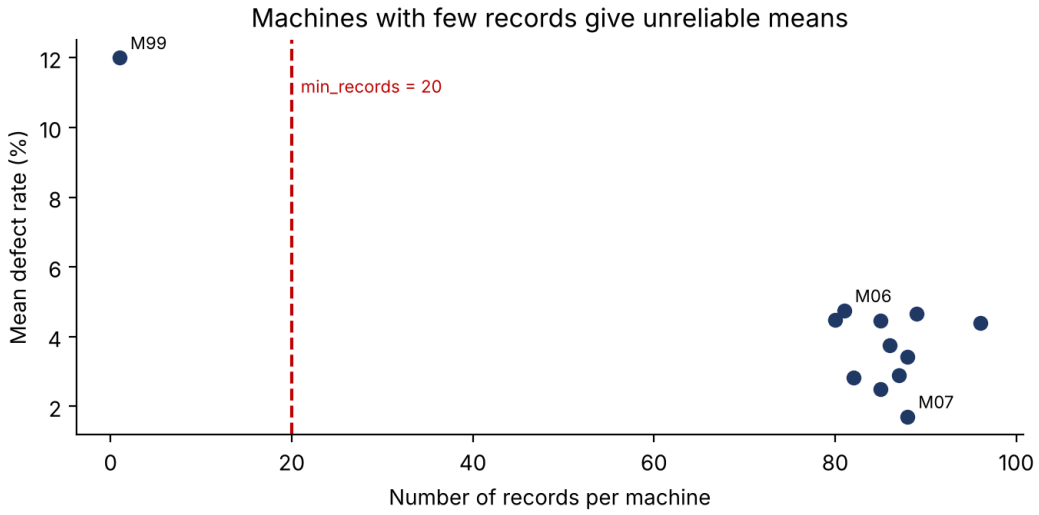
Funnel plot – why the minimum record rule is needed

Python code

```
g = original_ranking(clean_data(raw), "left")
fig, ax = plt.subplots(figsize=(7, 3.6))
ax.scatter(g["records"], g["mean_defect_rate"] * 100, color="#1F3864")
for m_, row in g.iterrows():
    if m_ in ("M99", "M06", "M07"): ax.annotate(m_, (row["records"], row["mean_defect_rate"] * 100), xytext=(5, 4), textcoords="offset points", fontsize=8)
ax.axvline(20, color="#C00000", ls="--"); ax.text(21, 11, "min_records = 20", color="#C00000", fontsize=8)
ax.set_xlabel("Number of records per machine"); ax.set_ylabel("Mean defect rate (%)")
ax.set_title("Machines with few records give unreliable means")
fig.tight_layout(); fig.savefig("figs/q7_funnel.png", dpi=200); plt.close(fig)
print("saved")
```

Output

```
saved
```



F. Justify the Final Design

Python code

```
print("Sensitivity of the redesign to the min_records threshold (left merge, Defects imputed):")
for k in [1, 5, 20, 40, 80, 85]:
    r, _ = robust_ranking(clean_data(raw), info, min_records=k)
    print(f"  min_records = {k:2d}: machines ranked = {len(r):2d}, top-1 = {r.index[0]}, top-5 = {list(r.index[:5])}")


summary = pd.DataFrame({
    "Measure": ["Scenarios with the same top-5", "Top-1 machine across scenarios", "Unmatched records reported",
                "Run time (ms)", "Machines ranked"],
    "Original": [f"{agree} of 4", "M99 or M06", "0 (silent)", f"{t_o*1000:.2f}", int(base[("left", "impute")].shape[0])],
    "Redesign": [f"{agree_new} of 4", "M06 always", "1 record + 1 machine", f"{t_n*1000:.2f}", int(new[("left", "impute")].shape[0])]})
print()
print(summary.to_string(index=False))
```

Output

```
Sensitivity of the redesign to the min_records threshold (left merge, Defects imputed):
  min_records =  1: machines ranked = 12, top-1 = M99, top-5 = ['M99', 'M06', 'M02', 'M08', 'M09']
  min_records =  5: machines ranked = 11, top-1 = M06, top-5 = ['M06', 'M02', 'M08', 'M09', 'M03']
  min_records = 20: machines ranked = 11, top-1 = M06, top-5 = ['M06', 'M02', 'M08', 'M09', 'M03']
  min_records = 40: machines ranked = 11, top-1 = M06, top-5 = ['M06', 'M02', 'M08', 'M09', 'M03']
  min_records = 80: machines ranked = 11, top-1 = M06, top-5 = ['M06', 'M02', 'M08', 'M09', 'M03']
  min_records = 85: machines ranked =  8, top-1 = M02, top-5 = ['M02', 'M09', 'M03', 'M01', 'M10']


                  Measure  Original          Redesign
 Scenarios with the same top-5    2 of 4             4 of 4
Top-1 machine across scenarios M99 or M06          M06 always
   Unmatched records reported 0 (silent) 1 record + 1 machine
            Run time (ms)      3.94               9.27
          Machines ranked       12                 11
```

Evaluation: The objective was achieved: stability rose from 2 of 4 to 4 of 4 scenarios, the top machine is M06 in every scenario, and the unmatched record and the machine without production are now reported instead of being lost silently. The prediction was confirmed: the means of the ranked machines are identical to the original (only the selection changed), and the run time roughly doubled (the ratio shown in the table above; a difference of only a few milliseconds), inside the accepted limit of 3×. The new costs are this extra run time, one extra table to read, and a judgement parameter, the threshold of 20 records. The sensitivity table shows the top-5 result is unchanged for every threshold from 5 to 80, so the conclusion does not depend on the exact value; only min_records = 1 brings M99 back, and a threshold as high as 85 starts to exclude genuinely well-measured machines (M06 has 81 records), so the threshold must stay well below a typical machine’s record count.

When the original may remain preferable: for a quick exploratory look at a master table that is known to be complete and where every group has a large number of records, the original two-line ranking is simpler and just as correct.